Design & Analysis of Algorithms

1) Analysis of Algorithms
   - → Algorithms Concept & Lifecycle
   - → Analysis of Algorithms
     - * Need for Analysis
     - * Performance Analysis: Time & Space Complexities
   - → Methodology of Analysis
     - * Aposteriori Analysis
     - * Apriori Analysis
   - → Types of Analysis
   - → Asymptotic Notations (ASN)
   - → Properties of Asymptotic Notations
   - → Framework for Analyzing Recursive Algorithms.
   - → Apriori analysis of Non-Recursive Algo
   - → Analyzing Loops
   - → Space Complexity
     - * Components of Space Complexity
     - * Example
   - → Maths topic (diff types of fns, * logarithms, * Summation series, Probability, * Exponents, Bionomial Coefficients, * Recurrences, Maths Induction)

2) Design Strategies:
   - → Divide & Conquer (D C)
     - * General Method
     - * Min-max Problem
     - * Merge Sort
     - * Binary Search
     - * Quick Sort

→ Matrix Multiplication
  ⇒ D and C Method
  ⇒ Strassen's Method
→ Long Integer Multiplication (LIM)
→ *Master Method for D and C Relations | substitute method for D&C
→ *Recursion Tree " " " " "
→ *Back Substitution method " " " "
  (Universal method)

3) Greedy Methods
→ General Method
→ Knapsack Problem
→ Job Sequencing with Deadlines
→ Optimal Merge Patterns
  * Huffman Coding
→ Minimum Cost Spanning Trees
  * Prim's Method
  * Kruskal's Method
  * Comparison of Prim & Kruskal method
  * Dijkstra - Kalaba Algo.
→ Shortest Path Problem
  * Dijkstra's Single Source Shortest Paths Algorithm

4) Dynamic Programming (DP)
→ Method
→ Diff b/w DP, Greedy Method & D&C
→ Multistage Graphs
→ Travelling Salesperson Problem (TSP)
→ Binary knapsack Problem (0/1)
→ All Pairs Shortest Paths (Floyd-Warshell)
→ Bellman-Ford Single Source Shortest Paths
→ Longest Common Subsequence (LCS)
→ Matrix Chain Multiplication (MCP) {non-square matrix}
→ Sum of Subsets (Decision Prob.)
→ Reliable System Design
→ Optimal Cost Binary Search Tree

5) Graph Algorithms
→ Representation of Graphs < Sequential: Adj. Matrix
Linked: Adj. List
→ Graph Traversals
→ DFS
* Undirected Connected Graphs
* Undirected Disjoint Graphs ; DFT
* Directed Graphs & Types of Edges
* DAG (directed acyclic graph)
↳ Topological Sort

→ BFS (Queue < ~~FIFO~~ LIFO Priority Queue

* ~~FIFO~~ BFS
* LIFO BFS → D-search
* LC BFS → Priority Queue.
→ Paranthesization Theorem
(classify types of edges)

6) Heap Algorithms & Priority Queue
→ Operations: Create, Insert, Delete, Modify
→ Applications: Heapsort
→

Book :-
Intro. to Algo. [Cormen (CLRS)]

Maths → • Logarithms
• Exponents
• Summation
• Differentiation & Integration
• Matrices

7) Hashing
→ Structure of Hash Table
→ Hashing Techniques :
→ Collision Resolution Technique
* Closed Hashing
* Open Hashing
* Comparison of Closed & Open Hashing Tech

8) Sets
→ Representations : (Tree & Array)
→ Operations
* Union
* Find

9) Sorting Algorithms
→ Basic Terminology.
→ Methods
* Bubble Sort
* Selection Sort → [Least no of swaps (W.C → O(n))]
* Insertion Sort
* Radix Sort → (Non-Comparison based)

10) Backtracking & Branch Bound
→ Basic Intro. [N-Queens Problem, Solve Sudoku, Rat in a Maze]
→ Terminology
→ Applications

DAA
1) DC
2) GM
3) DP → Problem solving
4) Backtracking
5) Branch & Bound

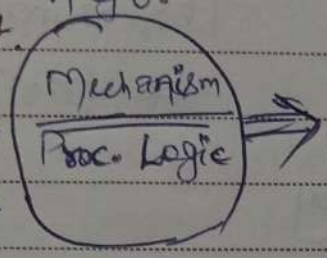
DAA → design & analysis of algorithm.

(a) Algorithm: Consists of finite set of steps/statement to solve a given Problem.

(set of steps for given prob.)

may accept 0/more input.

must produce at least one output.



→ one/more fundamental (basic/primitive oper^n's.

Op^n

Definite (clear) unambiguous

Effective (finite time)

Life cycles Steps :-

1) Problem Def^n
2) Requirements (SRS)
3) Design (Logic)
4) Express the Design → Algo / Flowchart (Algo. development)
5) Validation
6) Analysis
7) Implementation
8) Testing & Debugging

Need? (why) for Analysis

- Resource Consumption
  - Time (CPU)
  - Space (Memory)
- Performance Comparison [To check efficiency of algo]

PC

How? Methodology of Analysis

- Aposteriori Method (not preferred)
  - Experimentation <Platform Dependant>
    - H/W (CPU + mem)
    - S/W (OS + Prog. lang)
- Apriori Analysis (before) <Platform Independant>

Apriori Analysis

(i) Use some Pseudo language;
(ii) Computation model (RAM Model) within which algo. executes
(iii) Performance metric (fund. oper^n)
(iv) Characterize the Time w.r.t some framework (ASN)
(↓ oper^n → ↓ exit)

Apriori : Step count after

Order of magnitude of Statement

→ Frequency/count of the Basic/Primitive Op.

```
1  x ← x+y          } : 2
2  for i ← 1 to n   } : 1+n+1+n+
     for j ← 1 to n } : n+n(n+1)+nxn
       k ← k+3     } : nxn+nxn
```

} (4n² + 4n + 2) units

i) 1
ii) n2
(1+n²) = T(n)

Time : T(n) ≤ 2+(4n²+4n+2)

≤ (4n² + 4n + 4) units

mathematical f → shows Behaviour of fp

w.r.t input size (n)

ASN → ~~Asymtotic~~ Asymptotic notation

Tool to determine Bounds of fp

Upper, Lower, Tight

Big

Small/Little

i) Big-Oh (O): upper bound
2) Big-omega (Ω): Lower bound
3) Theta (Θ): tight bound

4) Small-Oh (o):
5) Small-omega (ω)

ASN

Time → f(n)

(upper bound, lower bound, Tight)

1) Big-Oh (O): upper bound of f" (some)

f(x) is O(g(x)) iff there exists ^ constants 'c' & n0 (c>0, n0>0)

such that $f(x) \leq c\,g(x)$ whenever $x > n_0$

$f(x) \Rightarrow O(g(x))$

ex :- $f(n) = n^2+n+1 = O(n^2)$

$n^2+n+1 < n^2+n^2+n^2$

$f(n) \le 3n^2, \quad n>1$

(c = 3, g = n^2)

* In determining the Bounds of the functions always use that fn $(g(n))$ which is Asymptotically tight to the given f^n.

2) Lower Bound / Big-omega (Ω):

$f(n)$ is $\Omega(g(n))$ iff $\exists c \& n_0$ such that $f(n) \ge c \cdot g(n)$ whenever $n > n_0$

ex :- $n^2+n+1 \Rightarrow \Omega(n^2)$

i) $n^2+n+1 > 1 \quad \therefore \Omega(1)$ ✓

ii) $n^2+n+1 > n \quad \therefore \Omega(n)$ ✓

iii) $n^2+n+1 > 1 \cdot n^2, \ n>1 \quad \therefore \Omega(n^2)$ more closer

(c = 1, g = n^2)

$f(n) = n + \log n$

$O(n) \quad \Omega(n)$

$f(n) = 2^{100}$

$O(1) \quad \Omega(1)$

$2^{100} < 3^{100} \cdot 1$ (c = 3, g = 1) $\qquad 2^{100} > 1 \cdot 100 \cdot 1$ (c = 1, g = 1)

3) Theta Notation : (Θ) : Tight Bound

$f(n)$ is $\Theta(g(n))$ iff

$f(n)$ is $O(g(n))$ & $f(n)$ is $\Omega(g(n))$

$$c_2 \cdot g(n) \le f(n) \le c_1 \cdot g(n)$$

$f(n) = n^2+n+1 \begin{cases} O(n)^2 \\ \Omega(n^2) \end{cases} \therefore \Theta(n^2)$

1) Smaller f^n are generally in the order of Bigger f^n's
$n < n^2$, $\log n > \log\log n$, $\frac{1}{n} < \log n$, $\sqrt{\log n} > \log\log n$

2) Bigger f^n's are omega of smaller f^n's.

3) If rate of growth of both f^n are same then Θ

Small Notations: Bounds (O/L) provided by Big Notations may or may not be tight
But Bounds provided by Little/Small notations is always NOT tight.

4) Small-oh (o):
$f(n)$ is $o(g(n))$ iff for all $c>0$ & n_0
$$f(n) < c \cdot g(n)$$
$n > n_0$.

5) Small omega (ω)
$f(n)$ is $\omega(g(n))$ iff for all $c>0$ & n_0
$$f(n) > c \cdot g(n)$$

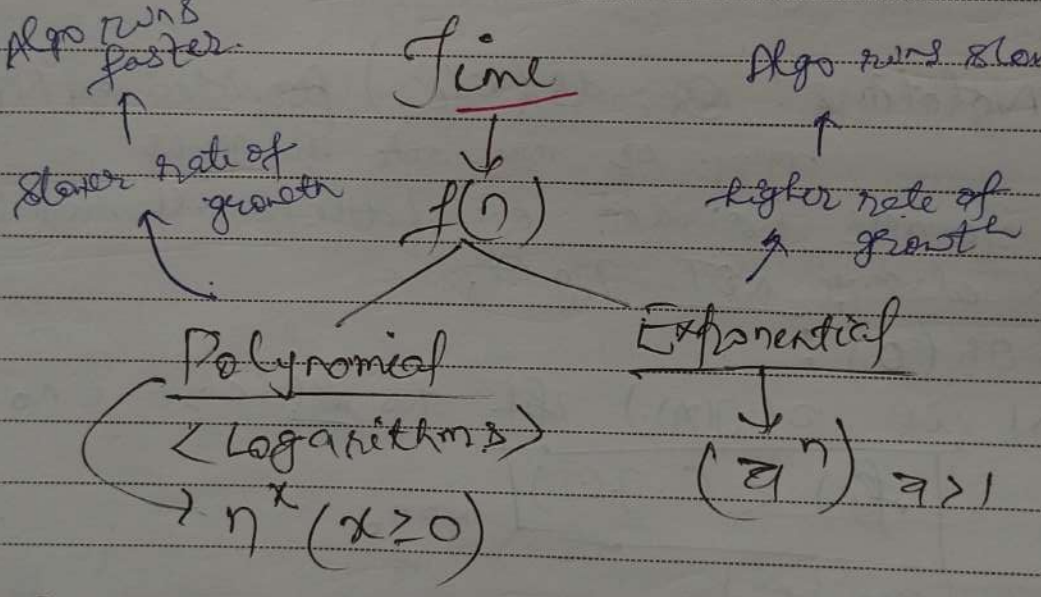
$f(n) = n^2$ — $\Omega(n^2)$ ✓ — $\Omega(n)$ ✓
$\omega(n)$ ✓
$\omega(n^2)$ ✗

polynomial are always in order of exponential &
Logarithms are " " " Polynomial

Analogy b/w ASN & Real No's

f, g : fn's ; a, b : Real no's

1) $f(n)$ is $O(g(n)) \iff a \leq b$
2) $f(n)$ is $\Omega(g(n)) \iff a \geq b$
3) $f(n)$ is $\Theta(g(n)) \iff a = b$
4) $f(n)$ is $o(g(n)) \iff a < b$
5) $f(n)$ is $\omega(g(n)) \iff a > b$



Dominance Relation

$c \underset{ASN}{<} \text{Log} \underset{A}{\leq} \text{Polynomial} \underset{A}{<} \text{Exponential}$

$c < \log n < n < n \cdot \log n < n^2 < n^3 < \cdots < 2^n < 4^n < n^n$

Arrange the f^n in increasing order of ratios of growth

1) n^2 ; $n \log n$, $n\sqrt{n}$, e^n ; n ; 2^n ; $(1/n)$

$\rightarrow \frac{1}{n} < n < n \log n < n\sqrt{n} < n^2 < 2^n < e^n$

2) 2^n ; $n^{3/2}$; $n \log n$, $n^{\log n}$

$n \log n < n^{3/2} (n\sqrt{n}) < n^{\log n} < 2^n$

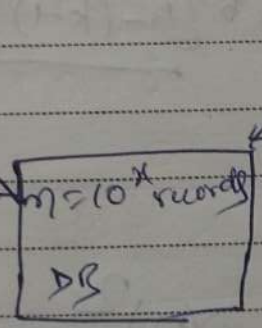
$\log n^{\log n}$ $\quad$ $\log 2^n$

$\log \log n$ $\quad$ $n \log 2$

$(\log n)^2 < n$

3) $n^{(1/3)}$; e^n ; $n^{7/4}$; $n \log^9 n$; 1.0001^n

Q



for which A outperformes B.

Let $n = 10^2$

A → $10 \cdot 10^2 \log 10^2 = 200$

B → $(10^{-4}) \times (10^2)^2 =$

$* \; n = 10^x$

$10 \cdot n \cdot \log n < 10.0001 n^2$

$10 (10^x) \log_{10} 10^x < 10^{-4} \times (10^x)^2$

$x \cdot 10^x \cdot 10 < \frac{10^{2x}}{10^4} \Rightarrow x < \frac{10^x}{10^5}$

$6 < 10^6 / 10^5$

$(6 < 10)$ A

$\boxed{x = 6}$

(Time Complexities for Recursive Algo.)

⇒ Recurrence Equation

```
Algorithm What(n)
{
  if(n==1) return;
  else {
    What(n-1);
    B(n);
  }
}
```

$O(n^2)$ — B(n) is $O(n)$

$T(n)$ = Time Complexity of What(n)

$$T(n) = \begin{cases} c, & n=1 \\ a + T(n-1) + n, & n>1 \end{cases} \quad [a, b > 0 \text{ constants}]$$

$$T(n) = T(n-1) + (n+a) \quad -(1)$$

$$= T(n-1) + bn \quad -(2)$$

$$T(n-1) = T(n-2) + b(n-1) \quad -(3)$$

$$T(n) = T(n-2) + b(n-1) + bn \quad -(4)$$

Back substitution

$n-k=1 \Rightarrow k=n-1$

$$T(n) = T(1) + (2+3+4+\cdots+n)\,b$$

$$= (c+2+3+\cdots+n)\,b$$

$$= (1+2+3+\cdots+n)\,b \quad \text{Let } b=1$$

$$= \frac{n(n+1)}{2} \cdot b \Rightarrow \frac{n(n+1)}{2} \leq O(n^2)$$

$$T(n-3) + b(n-2) + b(n-1) + bn$$

$$= T(n-k) + b[n-(k-1)] + \cdots + bx - 5$$

```
Algorithm A(n)
{
  if(n==2) return;
  else return A(√n);
}
```

- $A(\sqrt{n}) + n$ ↳ $a + T(\sqrt{n}) + b$
- $A(\sqrt{n}) + A(\sqrt{n})$ ↳ $a + 2\cdot T(\sqrt{n}) \rightarrow O(\log n)$

$$T(n) = \begin{cases} c, & n=2 \\ a + T(\sqrt{n}) \end{cases}$$

Using Back substitution method

$$T(n) = T(n^{1/2}) + a \quad -(1)$$

$$T(n^{1/2}) = T(n^{1/4}) + a \quad -(2)$$

$$T(n) = T(n^{1/4}) + 2a \quad -(3)$$

$T(n) = T(n^{1/2^2}) + 2a$ — (3)

$= T(n^{1/2^3}) + 3a$

$= T(n^{1/2^k}) + ka$

$n^{1/2^k} = 2$

$\frac{1}{2^k} \cdot \log n = 1$

$2^k = \log n$

$k = \log \log n$

should be equate to boundary cond^n

$= T(2) + a \log \log n$

$= c + a \log \log n \Rightarrow \theta(\log \log n)$

```
void abc (char *c)
{ if (*s1 != '\0')
  { printf ("%c", *s);
    abc (s+1);
    abc (s+1);
  }
}
```

s is a pointer to a string of length 'n' char
Then the T.C of abc : $O(2^n)$

$T(c) = c, \quad n = 1$

$= a + b + T(n-1) + T(n-1)$

$= 2 \cdot T(n-1) + d$

$\theta(3^n)$ ← exponential time

SPACE COMPLEXITY

Input + Work space

Work space = Computation + Additional space

In measuring space complexity we consider only the space requirement for work space (extra space) & Recursion stack

int t, workspace;

variable t = const. space $\theta(1)$

```
* Algorithm RSum (A, n)
{ if (n == 1) return (A[1]);
  else return [A[n] + RSum (A, n-1)];
}
```

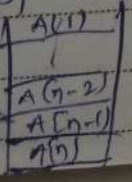
$T(n) = c, \quad n = 1$

$= a + T(n-1), \quad n > 1$

$O(n)$

T.C : $O(n)$; S.C : $O(n)$

space for recursion space :



(n)* activation record

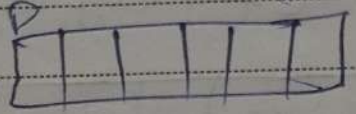
Divide & Conquer

: problem is said to be small, if it can be solved in one/2 basic operations.

Time Complexity = Divide & Conquer Recurrence rel^n

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ T(n_1) + T(n_2) + \cdots + T(n_k) + f(n) & \text{otherwise} \end{cases}$$

```
DandC(A, p, q)
{ if (SMALL(A)) return;
  else{
    m ← (p+q)/2
    S1 ← DandC(A, p, m);
    S2 ← DandC(A, m+1, q);
    Combine(S1, S2);
  }
}
```



$n = q - p + 1$

$T(c) = c, \quad n = 1 \text{ (small)}$

$= g + T(n/2) + T(n/2) + f(n)$

$T(n) = 2T(n/2) + f(n), \quad n > 1$ (else)

* Generalized DAC Recurrence

$$T(n) = a \cdot T(n/b) + f(n), \quad n > d$$

$$= c, \quad n \le d \text{ (small)}$$

No. of subproblems: $a \ge 1$

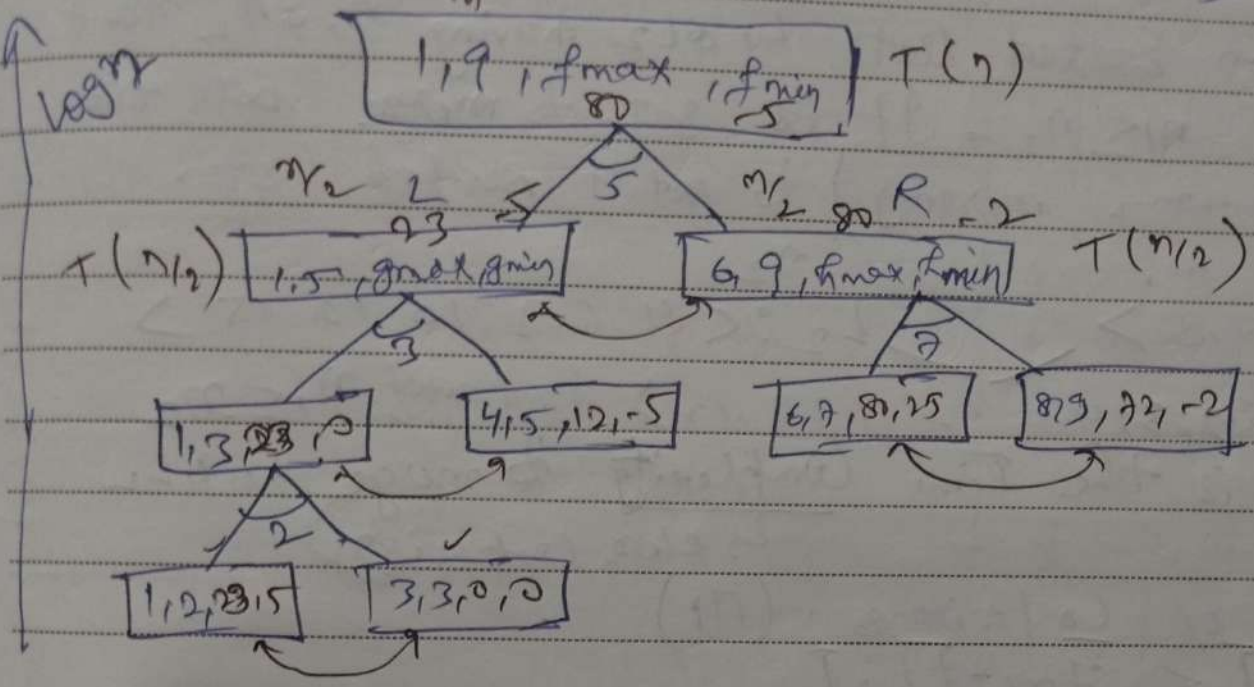
Size of each subproblem: $b > 1$

$f(n)$ is always +ve

Space ∝ Recursion stack

① Min-Max Mergesort

A: <23, 15, 10, -5, 12, 25, 80, 72, -2>



no. of elements comparisons for D&C — maxmin(n)

$$T(n) = \begin{cases} 0, & n = 1 \\ 1, & n = 2 \end{cases} \quad \text{small}$$

$$= 2 \cdot T(n/2) + 2, \quad n > 2$$

Back substitution ← Master method + $O(n)$

$T(n) = 2 \cdot T(n/2) + 2$

$T(n/2) = 2T(n/4) + 2$

$T(n) = 2(2T(n/4) + 2) + 2$

$= 4 \cdot T(n/2) + \sum_{i=1}^{2} 2^i$

$= 2^2 \cdot T(n/2^2) + \sum_{i=1}^{2} 2^i$

$= 2^k \cdot T(n/2^k) + \sum_{i=1}^{k} 2^i$

$= \frac{3n}{2} - 2$

For all cases

$S(n) = O(\log n)$

(IV) • D and C MergeSort
↳ Merging Process (conquer/combine)

Given, Two Sorted Lists L_1 & L_2 having n_1 & n_2 no of elements, $n_1 \leq n_2$. It is req. to merge both the lists (2-way merging) to get 1 sorted list.

$L_1 < 2, 3, 5, 8 >$ (n_1) $\quad$ $L_2 : < 4\ 6\ 9\ 10\ 12\ 15 >$ (n_2)

$L_0 : < 2, 3, 4, \ldots, 12, 15 >$ final sorted list. $n_1 \leq n_2$

∴ What is the Time Complexity to merge L_1 & L_2
↳ elem. comparison.

• Min. # of Comparisons : (n_1)
$|L_1| < \text{First} |L_2|$

• Max. # of Comparison : $(n_1 + n_2 - 1)$

$L_1 : < 2, 3, 5, 40 >$ $\quad$ $L_2 : < 8, 10, 12, 15, 18, 20 >$

$(n_1 - 1)$ $\quad$ $(n_1 - 1) + n_2$

$\therefore (n_1 + n_2 - 1) = O(n_1 + n_2) = O(n)$
Time.

Let $n = n_1 + n_2$

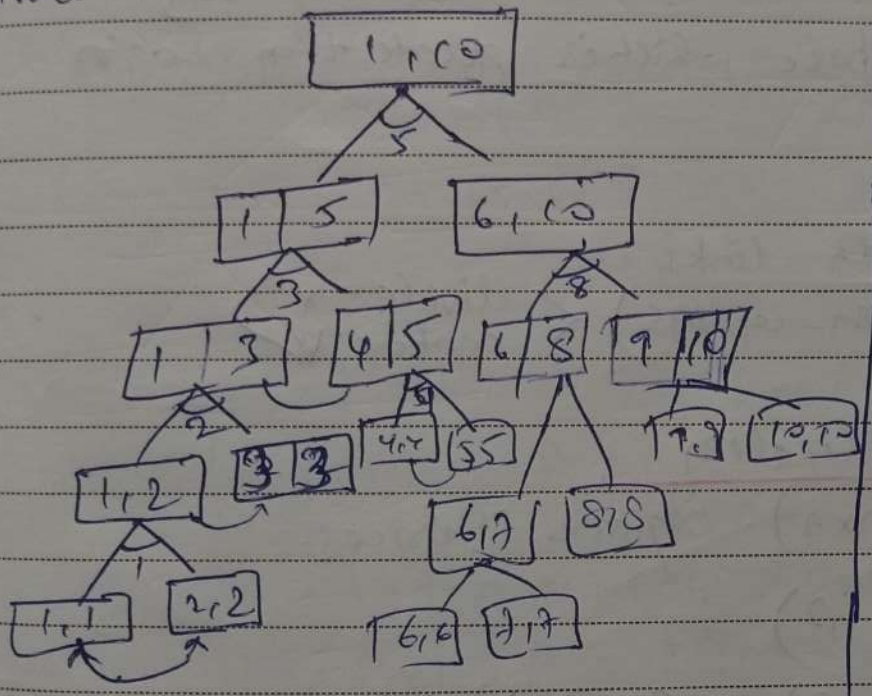
∴ In order to merge two sorted Lists L_1 & L_2 having n_1 & n_2 no. of elements, then the no. of elem. comp's oscillate b/w n_1 & $(n_1 + n_2 - 1)$

Merge Sort

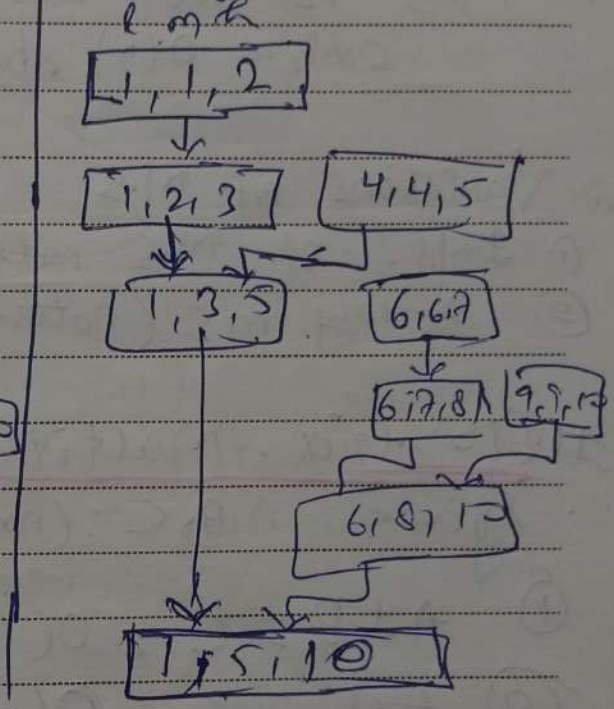
A: [3 1 0] [2 8 5] [1 7 9] [6 5 2] [3 5 1] [4 2 5 8 6 1 2 5 4 8 0 5 2 9]

B: 1 7 2 8 5 3 1 0 3 5 1 6 5 2

Divide Tree



Merge



Performance analysis of MS(n)

① Time Complexity :

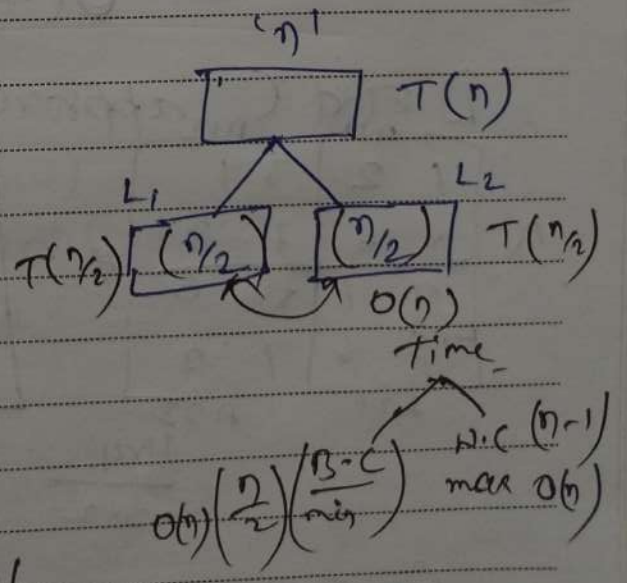
Let T(n) repr. the T.C of MS(n)

$T(n) = c, \quad n = 1$

$= 2T(n/2) + \{a+b+d(n)\}$

$= 2T(n/2) + d(n)$

$T(n) = 2T(n/2) + bn, \quad n \geq 1, \; b > 1$



B.S means R.Tree $= cn \log n + bn$

$T(n) = \Theta(n \cdot \log n)$

$= \Omega(n \cdot \log n)$

$T(n) = \Theta(n \log n)$

* Space complexity: array + depth of stack

↳ $(c + n + \log n)$
(c → const vals)

$\rightarrow O(n)$

→ M.S is NOT IN PLACE because it requires extra $O(n)$ space which is greater than $\log n$

* Variations of M.S

① Impl. of M.S with links

② 2-way MS (Bottom-up MS) < divide: x, combine: ✓ >

Matrix - Multiplication

Given. A, B, C : $(n \times n)$: Square Matrices.

① $A + B = C$: $O(n^2)$

② $A * B = C$: $O(n^3)$, S.C = $O(n)$

→ A_{ij}, B_{ij}, C_{ij} : $\frac{n}{2} \times \frac{n}{2}$ $\left(T\left(\frac{n}{2}\right)\right)$

D and C approach:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} * \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 1 & 5 & 6 \\ 5 & 6 & 1 & 2 \end{bmatrix} * \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

4×4 → 2×2

$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$ — ①

$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$ — ②

$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$ — ③

$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$ — ④

16×16 → 8×8 → 4×4 → 2×2 → 1×1

Let $T(n)$ be T.C of D and C - MATMULP($n \times n$)

$$T(n) = c \quad , n = 1 \; [n \leq 2]$$

$$\boxed{T(n) = 8 \cdot T\left(\frac{n}{2}\right) + bn^2} \quad , n > 1 \; (n > 2), \; b > 0$$

M-M ↳ $f(n)$ is $O(n^{\log_2 8 - \epsilon})$

n^2 is $\Omega(n^{3-\epsilon})$ $\epsilon = 1$

$\therefore T(n)$ is $\Theta(n^3)$

Space complexity ↳ qC : $O(\log n)$

* Strassen's Approach (D&C)

$$P = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$
$$Q = (A_{21} + A_{22}) \cdot B_{11}$$
$$R = A_{11} \cdot (B_{12} - B_{22})$$
$$S = A_{22} \cdot (B_{21} - B_{11})$$
$$T = (A_{11} + A_{12}) \cdot B_{22}$$
$$U = (A_{21} - A_{11}) \cdot (B_{11} + B_{12})$$
$$V = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

$$C_{11} = P + S - T + V$$
$$C_{12} = R + T$$
$$C_{21} = Q + S$$
$$C_{22} = P + R - Q + U$$

additional matrix → $P, Q, R, S, T, U, V : (n/2 \times n/2)$

like $A_{ij}, B_{ij}, C_{ij} : (n/2 \times n/2)$

7 - Multiplication : $(n/2 \times n/2)$

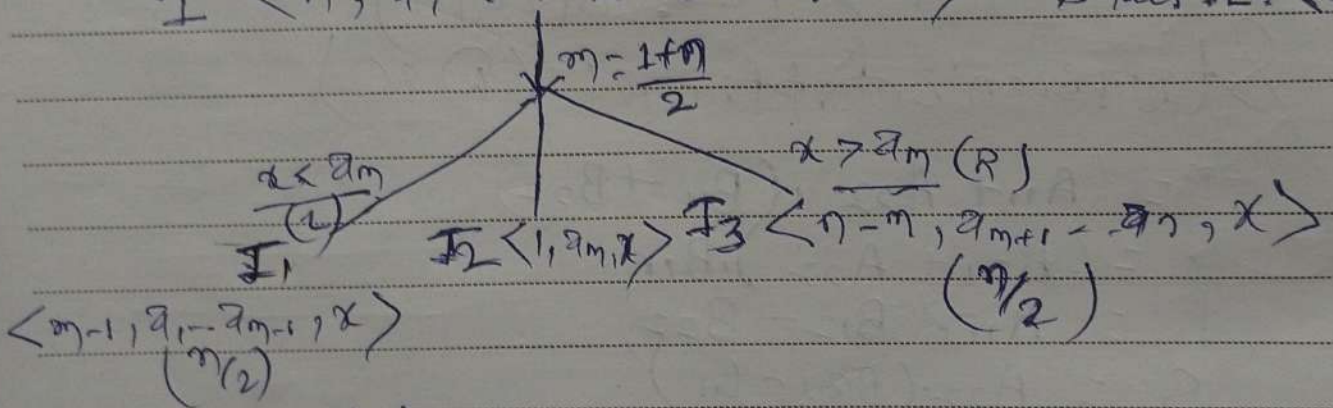
18 - Additions / Subtractions : $(n/2 \times n/2)$

$$T(n) = c \quad , n \leq 2$$
$$= 7 \cdot T(n/2) + 6n^2 \quad , n > 2$$

M.M. $f(n)$ is $O(n^{\log_2 7 - \epsilon}) = O(n^{2.81})$ $[\because \log_2 7 = 2.81]$

(3) Binary Search

$I \langle n, a_1, \ldots, a_n, x \rangle$ small : $L : \langle a \rangle$



$$T(n) = c \quad , n = 1$$
$$= a + T(n/2) \quad , n > 1$$
$$T(n) = T(n/2) + a = O(\log n)$$

* Problems :

i) Quick Sort : Partioning Process.

ii) Long Integer Multiplication : (LIM) →

(iii) No. of Inversions in an array.

* Master Method & Recursion Tree

Two long integers of n-digit bits each

a, b

① a ± b = c ; O(1)

② a * b = c ; O(1)

} int → 4B (6-digit)

S/W approach

```
int a[n], b[n], c[n];
for i ← 1 to n
    c[i] = a[i] ± b[i]
```

} : O(n) → to add 2 long integers

: O(n²) → to multiply two long integers

A. Karatsuba
AK optimisation

Q. Merge sort takes 308 — 64 elements sort in n·c

3608 — ? elements

Time Complexity of MS = (n · Log n) units

AS Time (Apriori time)

$64 \cdot \log_2 64 = (6 \cdot 64)$ units

6·64 units ∽ 308 (Aposteriori time)

$\frac{6 \times 64}{3} \left(\frac{30}{64 \times 6}\right)$ units ← 18

x

(4608)

(x units) (n)

x = n · Log n (n = 512)

4608

(M)

Greedy Method: is an Algo. Design Technique, used for solving Problems, whose solutions are viewed as a result of making a sequence of Decisions
The decisions are made in a step-wise (sequential) manner where each step applies Local optimality.

Terminology:

1. Problem Def^n
2. Constraints → Implicit (Local/Internal) → Feasibility criteria
   Explicit (Boundary of I/p)
3. Solution space: (Mathematical f^n)
4. Feasible sol^n: (satisfy implicit constraints)
5. Objective f^n: (Min/Max of a given criterion)
6. if (5) → Optimal sol^n: is that feasible sol^n that satisfies objective f^n.

Value → Unique (only one min/max)

Problems Classif^n

- Optimization → objective f^n → (Optimal sol^n)
- Decision → (Feasibility criteria) → Yes/No

- Greedy Method

↓

step-wise decision

↓

Local optimality: @ every step out of available options greedily. select that option which satisfies the objectivity & feasibility.

The min. Time to solve a Problem using Greedy approach is $O(n)$ when abstract procedure is of $O(1)$.

Types of Greedy Problem

- Subset Paradigm (Selecting of a subset of given Inputs) ex:- knapsack, JSD
- Ordering Paradigm ex:- MCST, optimal Merge Pattern, Tape storage

(I) KNAPSACK Problem: (Optimization + Subset Paradigm)

→ KNAPSACK (Bag) : Capacity of 'M'

→ 'n' — objects (O_i) $i = 1, n$

- Decision (x_i)
- Weight (w_i)
- Profit (P_i) (Maximize)

$\sum_{i=1}^{n} w_i \leq M$ → Trivial case (not imp.)

$\sum_{i=1}^{n} w_i > M$ (imp)

$\sum_{i=1}^{n} P_i$

KNAPSACK

x_i

$0 \le x_i \le 1$

Real / Fractional — Greedy

0/1 (Binary) — D.P

Problem Def^n:

O_i

x_i w_i P_i

$0 < x_i \le 1$

$(w_i x_i)$ $(P_i x_i)$

Objective fn — weight being put into the knapsack

Maximize $\sum_{i=1}^{n} P_i x_i$ subject to $\sum_{i=1}^{n} w_i x_i \le m$

where $0 \le x_i \le 1$

Explicit constraint

Implicit constraints (feasibility criteria)

$0 \le x_i \le 1$ (∞)

$\langle x_1, x_2, \ldots x_n \rangle$

- Sol^n Space of Real Knapsack : (∞)
- Sol^n " " 0/1 " : (2^n)

1) $n = 3$, $m = 20$; $\langle P_1, P_2, P_3 \rangle = \langle 25, 24, 15 \rangle$

$\langle w_1 w_2 w_3 \rangle = \langle 18, 15, 10 \rangle$

$\langle x_1 x_2 x_3 \rangle$

$\sum w_i > 20$

a) Greedy about Profit:

$x_1 = 1$ } $\sum w_i x_1 = 20$

$x_2 = 2/15$ } $\sum P_i x_i = 25 + \frac{2}{15} \times 24 + 0 = 28.2$

$x_3 = 0$

Object with highest profit may have high weight.

b) Greedy about Weight:

$x_3 = 1$
$x_2 = 10/15$
$x_1 = 0$

$\sum W_i x_i = 20$

$\sum P_i x_i = 15 + \frac{2}{3} \times 24 + 0 = 31$

c) Greedy about P/W : (Ideal)

$W_i \longrightarrow P_i$: O(1)

$1 \longrightarrow P_i/W_i$

∴ Dec order of P/W's,

$\frac{P_1}{W_1} = \frac{25}{18} = 1.38$

$\frac{P_2}{W_2} = \frac{24}{15} = 1.6$

$\frac{P_3}{W_3} = \frac{15}{10} = 1.5$

$x_2 = 1$
$x_3 = \frac{5}{10}$
$x_1 = 0$

$\sum W_i x_i = 20$

$\sum P_i x_i = 24 + \frac{1}{2} 15 + 0 = 31.5$

Q

Sl no.	Weight (kg)	Value (Rs)
1	10	60
2	7	28
3	4	20
4	2	24

Profit P_i

M = 11 (max weight), split items

$V_{opt} = 60$ (optimal algo)

V_{greedy} : $\frac{P_1}{W_1} = 6$, $\frac{P_2}{W_2} = 4$, $\frac{P_3}{W_3} = 5$, $\frac{P_4}{W_4} = 12$

- sort items by value to weight in descending order

$x_4 = 1$
$x_3 = 1$
$x_2 = 0$
$x_1 = 0$

$24 + 20 = 44$

$V_{greedy} = 44$

∴ $V_{opt} - V_{greedy} = 60 - 44 = 16$ Ans

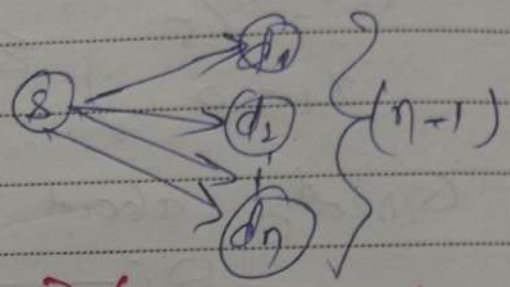
13-Oct-2024

VI - (multi-stage graphs)

Shortest Path Problem

(i) Single Pair Shortest Path : (s) ⟶ (d)
↳ Trivial prob. ☹

(ii) Single Source Shortest Paths :

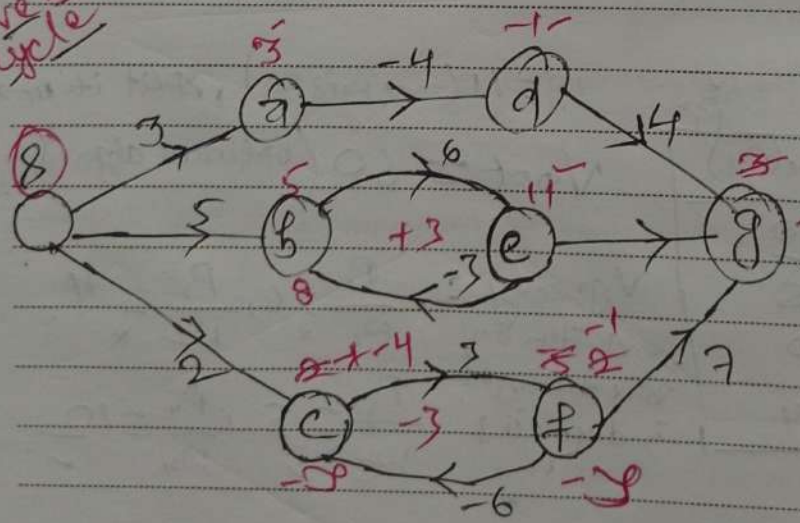


[Dijkstra's Algo.] (+ve weight edge.) ⟨Greedy strategy⟩

[Bellman-Ford] (-ve weight edge also) [Dynamic Programming] [No -ve wt. cycle]

(iii) All-Pair Shortest Paths (D.P)
(Floyd-Warshall's Algo.)

8. -ve wt. cycle



(sg) 5 8 4

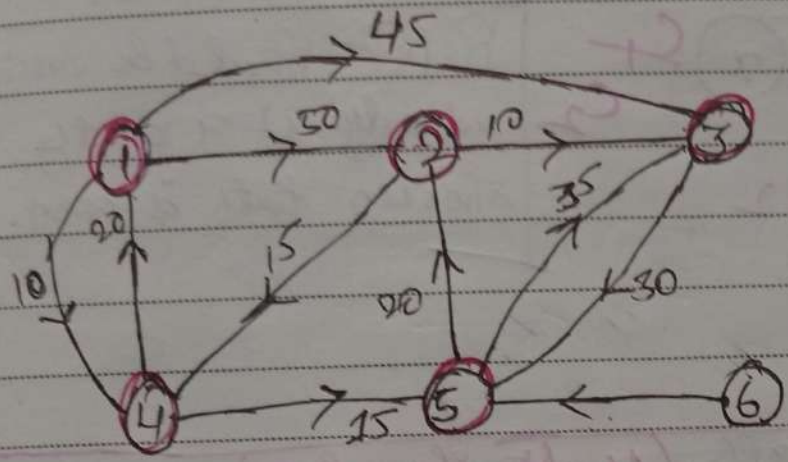
(sbg) 11 14

(sc) 2 -1 -4

(scf) 5 2 -1

→ In case of the graph has -ve wt. cycle reachable from source, then the vertices in the cycle & those vertices that are reachable through the cycle, shortest Paths are not possible.

Single Source Shortest Path - Dijkstras Algorithm



Dest / Src	1	2	3	4	5
1	-	50	45	10	∞
2					
3					
4					
5					
6					

c[i,j] : edge cost

Src Vertex : $v_0 = 1$

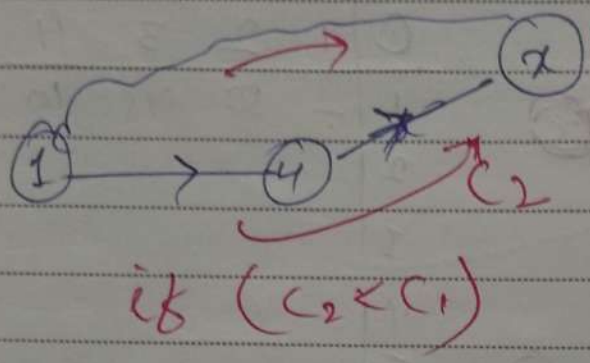
(i) Matrix Approach :

Vertex (Set) Selected	Dest: 1	2	3	4	5	6
	d- value:					
{1}	-	50	45	10	∞	∞
{1,4}	-	50	45	10	25	∞
{1,4,5}	-	45	45	10	25	∞
{1,4,5,2}	-	45	45	10	25	∞
{1,4,5,2,3}	-	45	45	10	25	∞ (unreachable)

d(x) : Cost of the Path from Src (v_0) to dest-vertex 'x'

disadv. : Path is not shown.

Relaxation Process: only for those vertex which are not selected so far.

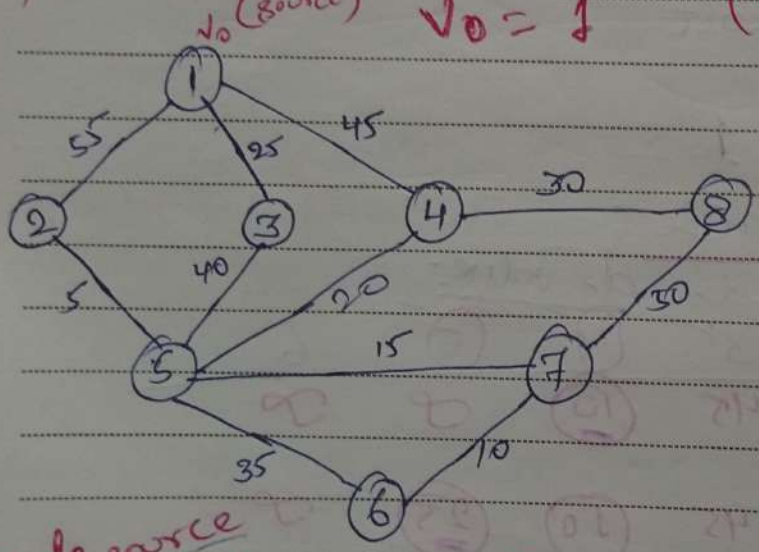


Relx² should be carried out only if a strictly shorter path is seen.

if $(C_2 < C_1)$

---

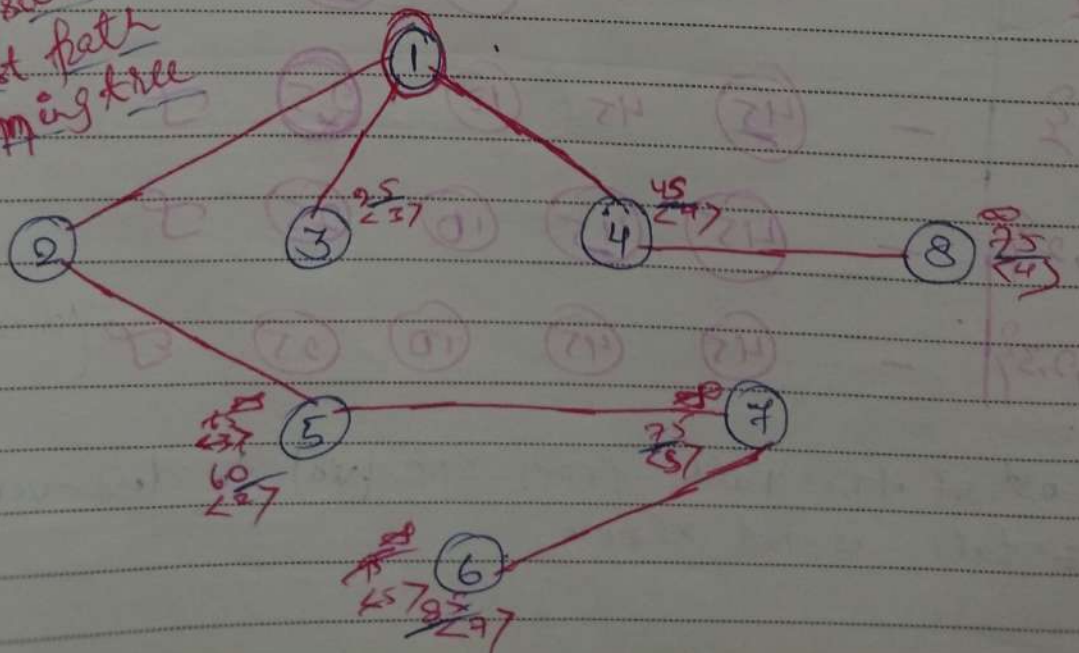
ii) Spanning Tree Approach - (It also shows path)



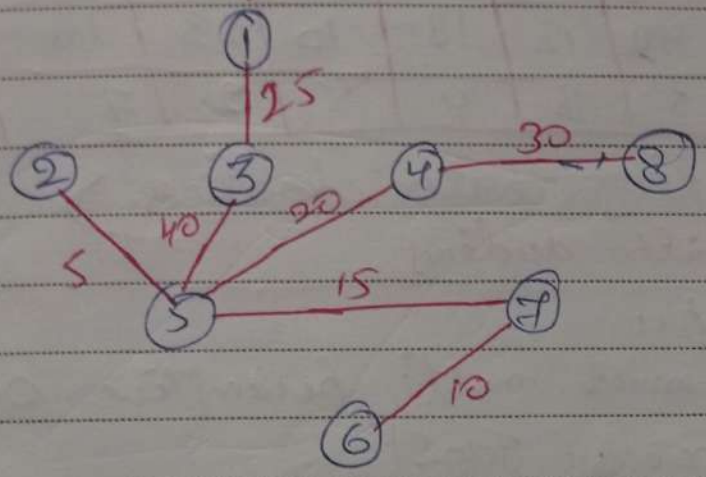
c → d-values → x

↳ select vertex having least d-value.

• Single source shortest path spanning tree

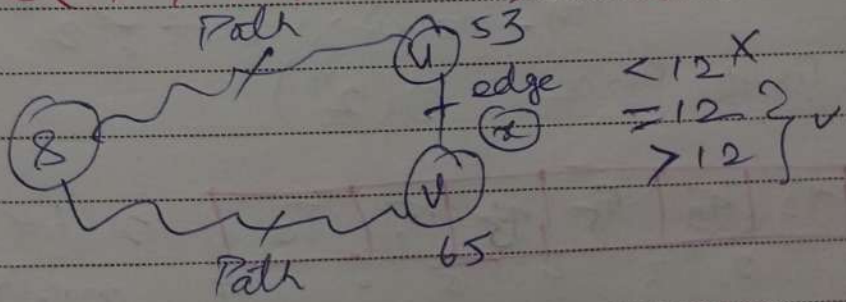


• Min-Cost Sp. Tree (By Prim's algo)



Q Consider a weighted graph. Let 'uv' be an edge in the graph. It is reported that the shortest path from source 's' to 'u' is 53 and shortest path from 's' to 'v' is 65; what is true w.r.to the edge cost (u,v)?

a) $C(u,v) \leq 12$ ✗
b) $C(u,v) < 12$ ✗
c) $C(u,v) \geq 12$
d) $C(u,v) > 12$



Q We are given 9 tasks $T_1, T_2, \ldots, T_9$. The execution of each task requires one unit of time. We can execute one task at a time. Each task T_i has a profit P_i & a deadline d_i. Profit P_i is earned if the task is completed before the end of the Deadline.

JSD

Task	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9
Profit	15	20	30	18	18	10	23	16	25
Deadline	7	2	5	3	4	5	2	7	3

JSD (CPU Scheduling with deadlines)

Job scheduling with deadlines

$O(n^2)$ → Single CPU
→ Non-Programmed multi preemptive OS
→ 'n' – Process (Jobs)

A.T	B.T	Deadline	Profit
'0'	1	d_i	P_i

→ "Select a subset of given (n) jobs, s.t the jobs in the subset are completable within their deadlines & maximize the Profit."

Size of sol^n space of JSD: 2^n
'n' – no. of subset ≥ 0

Brute force : $O(2^n)$

Ans →

CPU	T_2	T_7	T_3	T_5	T_4	T_1	T_8	= (147)
	0	1	2	3	4	5	6	7

max. Profit

→ discarded jobs are T_4 & T_6

14 Oct-2024

Dynamic Programming (DP): Richard Bellman

(XII)

It is a Technique used for solving Problems whose solutions are viewed as a result of making a sequence of decisions.

For some Problems, these seq. of Decisions are made in a step-wise manner & never making any ~~enormous~~ erroneous decisions. This is true for problems solvable by Greedy technique.

For many other problems, making stepwise decisions does not yield optimal solutions.

prob:- [coin-change, 0/1 knapsack, travelling sales person, multi-stage graph]

① COIN CHANGE PROBLEM

Given a set of coin values, construct a target sum money using as few coins as possible. You can use each coin value any no. of times.

Coins = $\{c_1, c_2, c_3, \ldots, c_n\}$

Target sum = M

ex:①

M=12, coins = {1, 2, 5}

Greedy Method

Sol²: [5+5+2] = 12

optimal (3 coins)

ex:②

M=6, coins = {1, 3, 4}

Greedy Method: {4+1+1} = 6

not-optimal → (3 coins)

optimal sol²: {3+3} = 2 coins

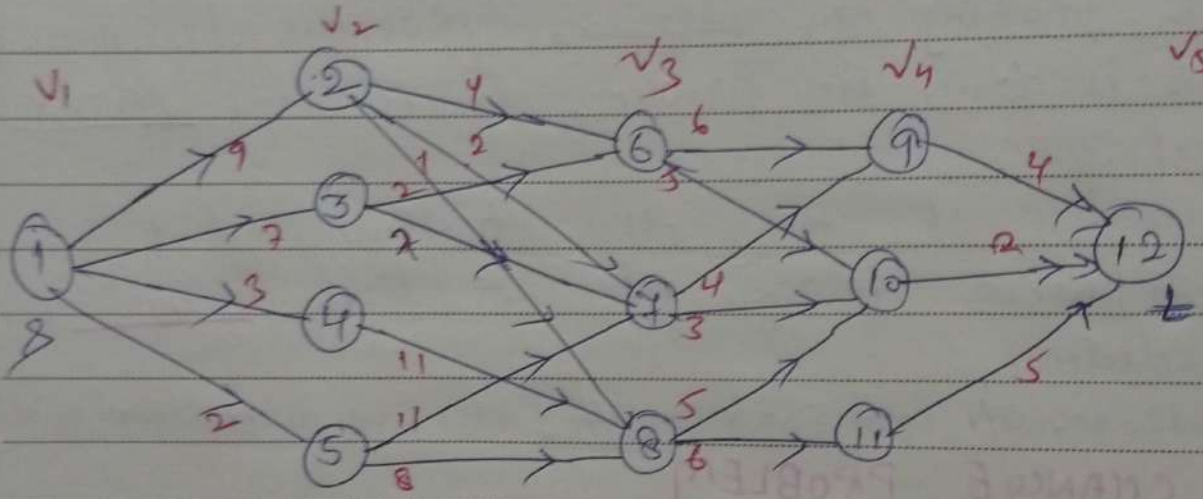
Greedy method fails

* Multi-Stage Graph (MSG)

$G = (V, E)$, $|V| = n$, $|E| = e$

(k-stage Graph) $(V_i \rightarrow V_{i+1})$

(Single-pair Shortest Path Problem)



Greedy Method: $\langle 1-5-8-10-12 \rangle$: 17

(for greedy method fails) Path: $\langle 1-2-7-10-12 \rangle$: 16

Alternate Sol?

① Brute Force (Enumeration): Enormous Time & Space Complexities; O(Exponential)

② Dynamic Programming (D.P)

→ Curtail (cut-off) those decisions sequences from where there is no possibility of getting optimal sol?

→ Store (Table) the results of sub-Problems (overlapping) so as to avoid recomputation.

Two elements of D.P

① Optimal Sub-structure Property [Principle of Optimality]

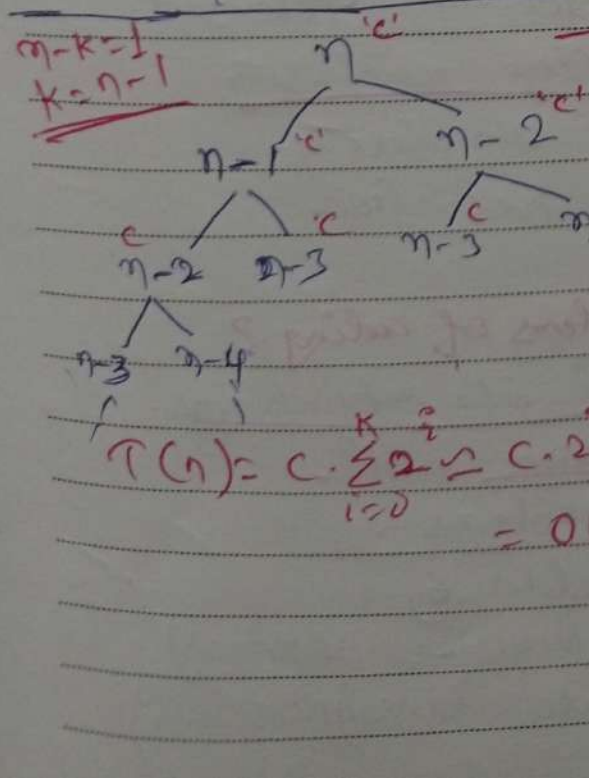
② Overlapping Subproblems

n^{th} Fibonacci No: <0, 1, 1, 2, 3, 5, 8, ---->

$$Fib(n) = \begin{cases} 0, & n=0 \\ 1, & n=1 \\ Fib(n-1) + Fib(n-2) \end{cases}$$

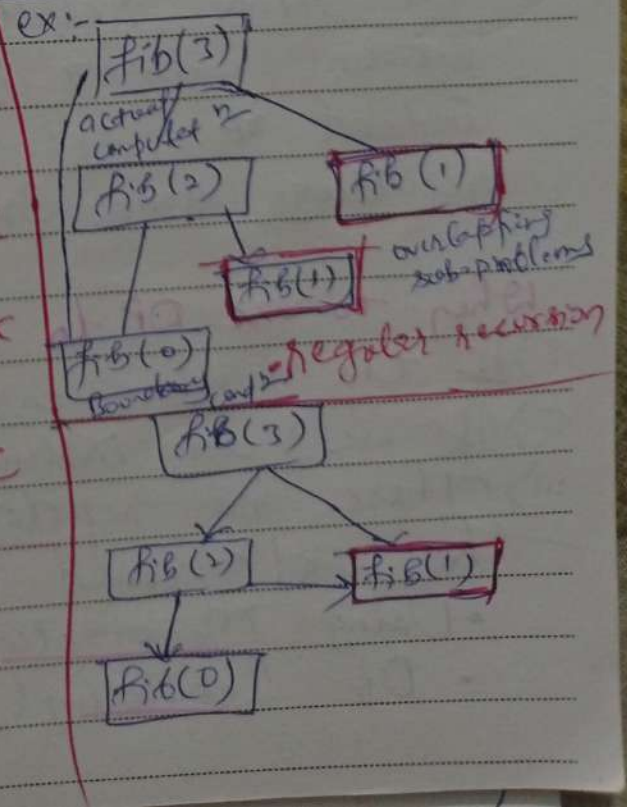
```
algo fib(n){
    if(n<=1) return(n);
    else {  a ← fib(n-1);
            b ← fib(n-2);
            return a+b; }
}
```

T.C = $O(2^n)$



$$T(n) = c \cdot \sum_{i=0}^{k} 2^i \simeq c \cdot 2^n$$

$$= O(2^n)$$



Dynamic Programming

- Top-down approach <Memoization> (Recursive Impl.)
- Bottom-up <Tabulation> (Iterative)

→ It is an optimization technique used to solve recursive problems more efficiently.
→ It is mainly done by finding if there are any repetitive subproblems and if so we compute it only once and store the value. By doing that, we can avoid re-computing repetitive values and save lots of time.
→ The final result is we get an optimized solution with a polynomial time complexity instead of exponential time complexity resulting from Brute force recursion.

Why to use DP to solve problems of coding?

i) If the problem can be divided into subproblems.
ii) If we can implement a recursive sol^n.
iii) There are repetative sub-problems.

The sol^n could be done either by:
- Using Memoization (top-down recursive approach)
- Or Tabulation (bottom-up iterative approach).

1- Memoization (Recursive, Top-down approach):

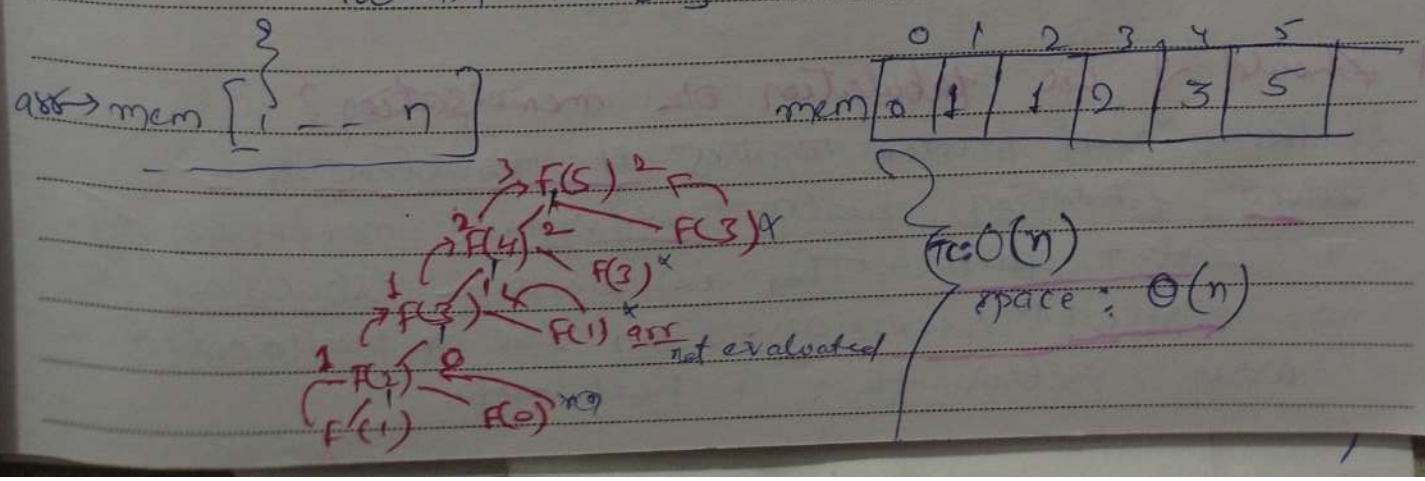
In this technique, we are still solving the problem recursively. But this time we are going to cache the result of each sub-problem in order not to solve it again repeatedly.

When we encountered the same sub-problem that we did before we just access its results from the cache instead of recomputing it again.

To make things clearer let implement the recursive brute force for Fibonacci numbers, then try to memorize it.

Memoization Impl. of nth Fib(n):

```
memFib(n) {
    if (mem[n] is undefined)
        if (n<2) result = n
        else result = memFib(n-2) + memFib(n-1)
        mem[n] = result
    return mem[n]
}
```



2. Tabulation (Iterative, bottom-up approach):

→ In this approach, we are solving the problem iteratively, we do this by filling a table with the results of the subproblems from the bottom till we reach our final result.

→ Each value is depending on the previous values in the table either direct or indirect way.

Tabulation implementation of nth Fib(n):

```
mem[1...n]
tabFib(n){
    mem[0] = 0
    mem[1] = 1
    for i = 2...n
    mem[i] = mem[i-2] + mem[i-1]
    return mem[n]
}
```

Time : O(n) , Space : O(n)

Should I use tabulation or memoization?

If the original problem requires all subproblems to be solved, tabulation usually outperforms memoization by a constant factor. This is because tabulation has no overhead for recursion. & can use a preallocated array rather than a hash map.

If only some of the subproblems needs to be solved for the original problem to be solved, then memoization is preferrable since the subproblems are solved lazily, i.e. precisely the computations needed are carried out.

D-P

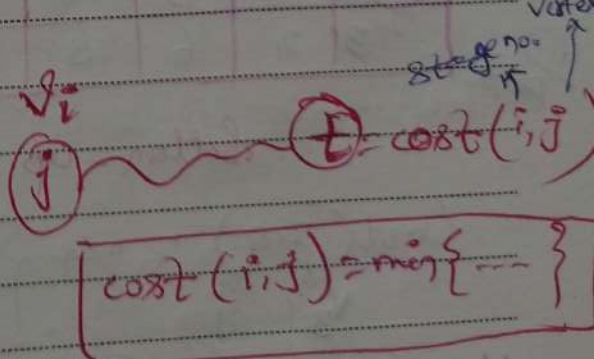
→ Evolves on the lines of B-F
→ Makes Decision by appealing to Principle of optimality.
→ Optimal substructure property
→ Overlapping subst.

Top-down Bottom-up

Q.1) MSG graph on prev page → App? : flow of gases in pipelines, fluids.

C	1	2	3	4	-	-	12
1	-	9	7				
2							
3							
i							
12							

C[i, j] = edge cost



Path cost: 8 → t

Let cost(i,j) repr. cost of the path from vertex 'j' present in stage 'i' to reach the destination vertex 't'

(8) $Cost(1,1) = \min\{c(1,K) + cost(2,K)\}$

$c(1,k) + cost(2,K)$

cost

1 — K — t

remaining cost min

$K \in V_2$

$\langle 1, K\rangle \in E$ → belongs to an edge

$$COST(i,j) = \min_{K \in V_{i+1}}\{c(j+K) + COST(i+1,K)\} \quad \text{--- (i)}$$

vertex

$\langle j, K\rangle \in E$ (must be an edge)

$COST(i-1,j) = c(j,t)$ — (2)

$D(i,j) =$ 'K' that Min. Eq (1)

Vertex	1	2	3	4	5	6	7	8	9	10	11	12
cost	16 min 2/3	7	9	18	15	7	5	7	4	2	5	-
D path	7	6	8	8	10	10	10	12	12	12	12	

Bottom-up approach

$Cost(7,8) = \min\{c(5+2), c(6+5)\}$

V_3 V_3

Construct the path

1 — 2 — 7 — 10 — 12 or 1-3-6-10-12

D(1,1) = 2 , D(4,10) = 12

D(2,2) = 7

D(3,7) = 10



1) Longest Common Subsequence (LCS) : String

1) X = <GGATAATTGAGA>

Substrings:	Subsequence:	Substring	Subsequence
1) CGA ✓	1) CGA ✓	• group of one/more continuous char's taken in a sequence.	• group of one/more characters that need not be contiguous nevertheless must be taken in order.
2) GATA ✓	2) GATA ✓		
3) AATTG ✓	3) AATTG ✓		
4) AAAAX ✗	4) AAAA ✓		
	5) TTTGA ✓	$= n \to \frac{n(n+1)}{2}$	$O(2^n)$
	6) (AGACT) ✗	$O(n^2)$	

✱ Common subsequence: optimization problem

2) X = <ABCBDAB> , Y = <BDCABA>

<BCB> = 3 ✓ common sub. seq of both X & Y.

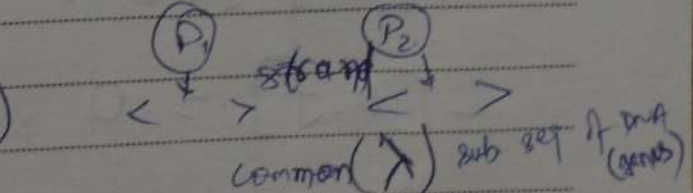
<BCBA> = 4 ✓ max. common subseq. (longest common subseq.)

<BD> = 2 ✓

Q → Given two strings 'x' & 'y' of length 'n' & 'm' problem statement: Char's the problem of LCS is to determine a common subseq. of X & Y of longest length.

✱ Applications:

1. Genomics (Genetic engg)
2. Plagiarism
3. Open source s/w testing
4. Search Engine



Case I:

X = <GTTCCTAAT A> (1 2 3 4 5 6 7 8 9 n)

Y = <CGATAATTGAGA> (1 2 3 4 5 6 7 8 9 10 11 12 m)

Case II:

X = <GTTCCAATA>

Y = <CGATAATTGAG>

Let, $X = \langle x_1, x_2 \ldots x_n \rangle$ (1 2 3 ... n) ; $Y = \langle y_1, y_2, y_3 \ldots y_m \rangle$ (1 2 3 ... m)

Derivation:

Let, $L[n,m]$ represent the length of common subsequence of the two strings $X(n)$; $Y(m)$;

* Case I : $X[n] = Y[m]$

$$L[n,m] = 1 + L[n-1, m-1]$$

* Case II : $X[n] \neq Y[m]$

$$L[n,m] = \max \{ L[n, m-1], L[n-1, m] \}$$

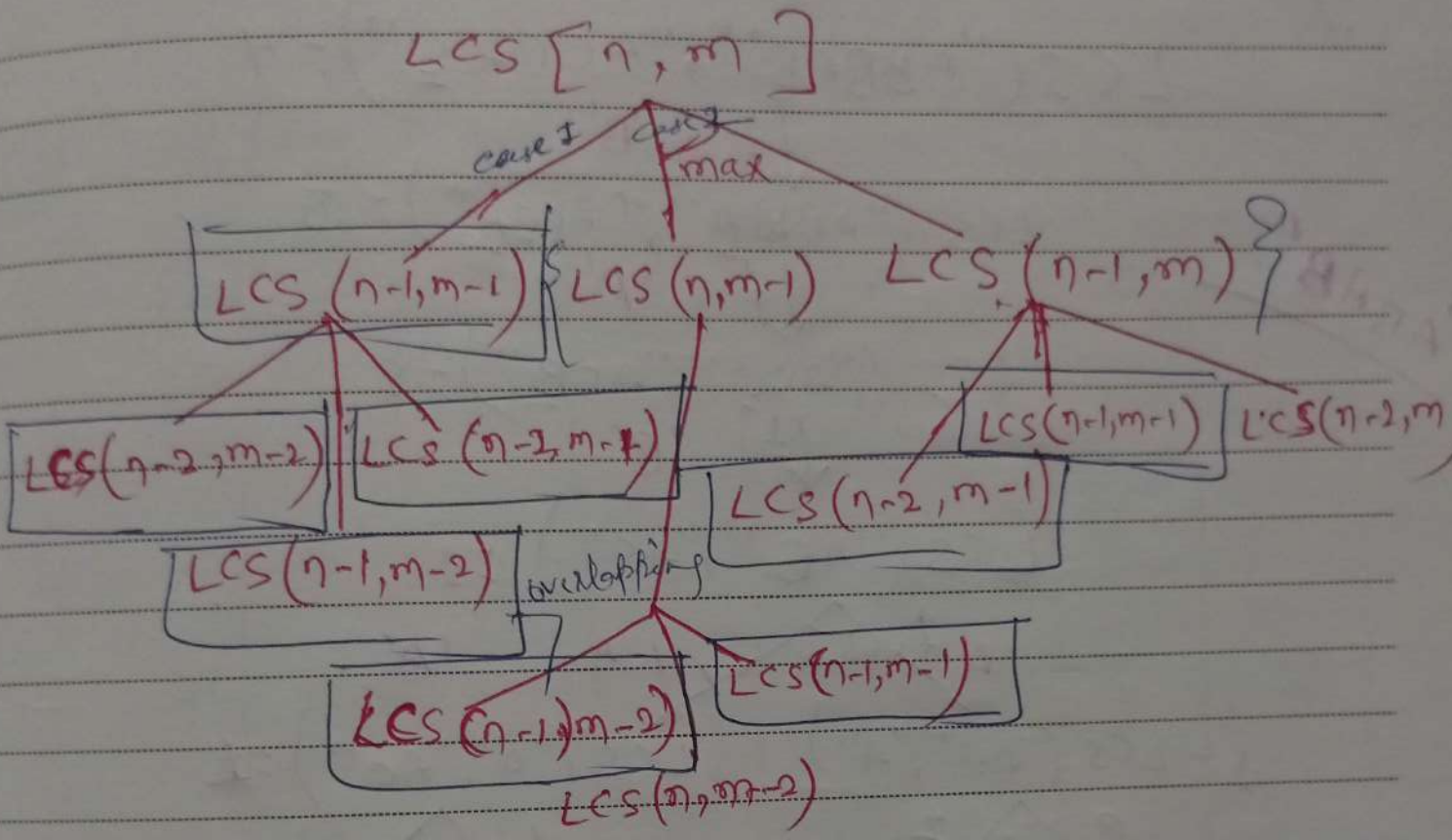
optimal sub-str:

* Boundary cond^n or terminating cond^n

$L[i,j] = 0$, $i = 0$ (or) $j = 0$

$L[0,j] = 0$, $j = 0, 1, 2 \ldots$

$L[i,0] = 0$, $i = 0, 1, 2, \ldots$



this tree shows that LCS has overlapping sub-problems, LCS has optimal sub-structure property hence, we can solve using principle of optimality (DP)

Blanked.

LCS("ABBAB", "ACBAB") = 4

'B'

LCS("ABBA", "ACBA") = 3

"ABAB"

'A'

LCS("ABB", "ACB") = 2

'B'

LCS("AB", "AC") = 1

A 1 max 1 A 'A'

1 = LCS("AB", "A")

LCS("A", "Ac") = 1

0 max 1

1 0

0 = LCS("AB", "") LCS("A", "A")

LCS("A", "A") LCS("", "Ac")

LCS("", "") = 0

LCS("", "") = 0

Tabulation process

Y[]	X[] →	0	1	2	3	4	5	6	7
↓		∅	M	Z	J	A	W	X	U
0	∅	0	0	0	0	0	0	0	0
1	X	0	0↑	0↑	0↑	0↑	0↑	1	1
2	M	0	1 →	1 →	1	1	1	1	1
3	J	0	1	1	2	2	2	2	2
4	Y	0	1	1	2	2	2	2	2
5	A	0	1	1	2	3 →	3 →	3	3
6	U	0	1	1	2	3	3	3	4
7	Z	0	1	2	2	3	3	3	4

Performance:

① Brute Force: Enumerate all possible subsequences of either X or Y & compare them with others:

$O(2^n \cdot m)$ or $O(m 2^m \cdot n)$ → exponential

② D.P (Tabulation):

T.C : $O(n \cdot m)$

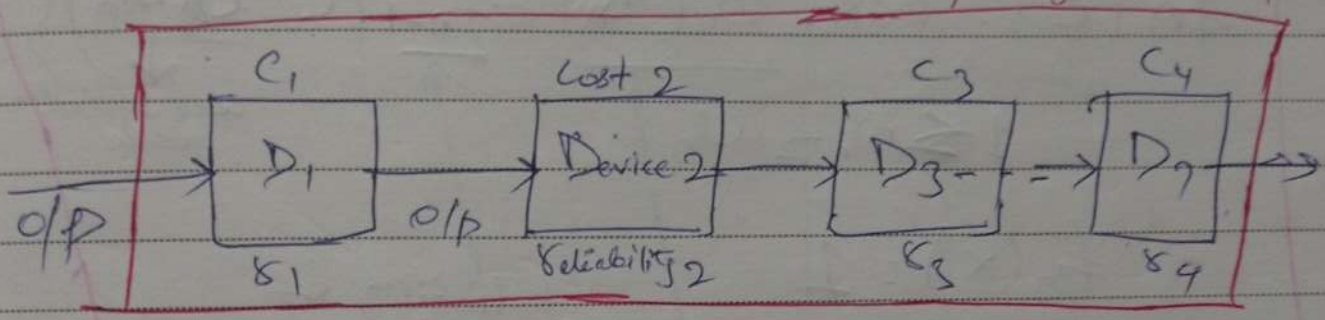
S.C : $O(n \cdot m)$

```
Algo LCS(X, Y)
  X[1..n], Y[1..m], L[1..n+1, 1..m+1]
{ 1.  for i ← 0 to n
      { for j ← 0 to n                    } O(n.m)
        { if((i=0) or (j=0))              }
            L[i,j] = 0
        else
            if(X[i] = Y[j])
                L[i,j] = 1 + L[i-1, j-1]
            else
                L[i,j] = max{L[i-1,j], L[i,j-1]}
      }
}
```

Cost → C
Reliability → r
↓
prob. of device functioning properly.

(3) Reliable System Design (Reliability Design)

design of an n-stage system :

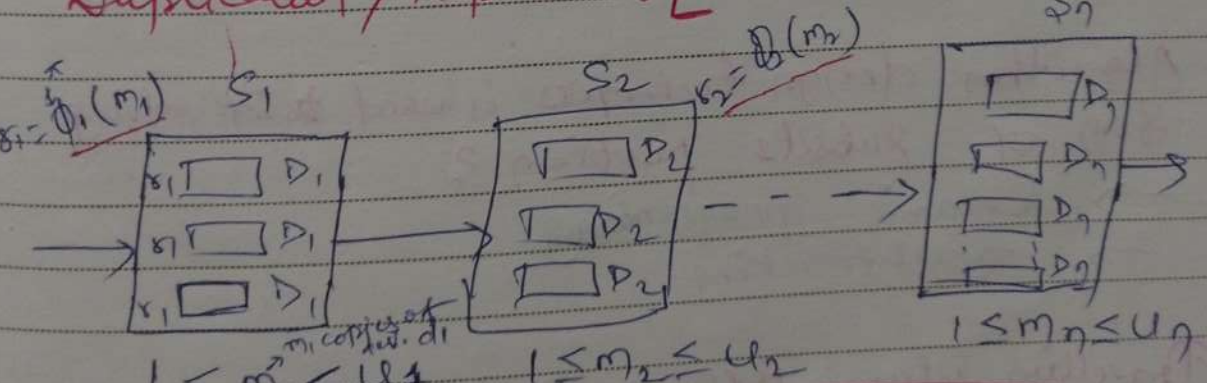


$0 \le r_i \le 1$

Total cost $= \sum_{i=1}^{n} C_i$

Reliability of System $= \prod_{i=1}^{n} r_i$ (product of reliability)

Duplication / Replication [improving reliability of system]



If r_i has m_i copies of D_i

→ Total cost of System $= \sum_{i=1}^{n} c_i \cdot m_i$

→ Reliability of System $= \prod_{i=1}^{n} \phi_i(m_i)$

$\phi_i(m_i) = 1 - (1 - r_i)^{m_i}$

prob. of success. → prob. of failure

Let $f_n(c)$ be the Reliability of n-stage System with total cost 'c'.

$$f_n(c) = \max \{ \phi_n(m_n) * f_{n-1}(c - C_n m_n) \}$$

$$1 \le m_n \le U_n$$

$$f_0(x) = 1$$

$\phi_n(m_n)$ → Rel. of nth stage

f_{n-1} → Reliability of (n-1) stages

$C_n m_n$ → cost of nth stage

Algorithms — Design Paradigms

Kruskal → Greedy
Quick Sort → divide & conquer
Floyd-Warshall → DP

Algorithm design technique is used in solving Sum of Subsets problem ?
→ Dynamic programming
→ Backtracking

Travelling Salesman Problem → D-P, Branch & Bound

0/1 KNAPSACK → D-P, Backtracking, Branch & Bound (BB)

Graph Techniques & Components

Graph : $G=(V,E)$ $|V|=n$, $|E|=e$

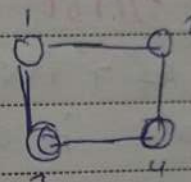
→ Representation

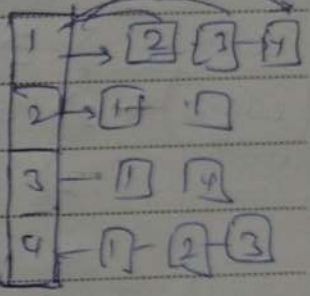
(Sequential rep^2) Adjacency Matrix | Adjacency List (Linked rep^2)

• size → $n \times n$

T.C $O(n^2)$

A	1 ---- n
1	
2	A[i,j] = 0/1
:	
n	





• undirected graph

• size = $\frac{n+2e}{O(n+e)}$

• T.C = $O(n+e)$

→ if graph is weight then entries will bare the weight present b/w i, j.
else entries will be 0/1.

* From both which one to choose depends on Graph:

Graph

Complete Dense | Sparse

Complete Dense → Matrix | Adj. List

Sparse → Adj. List : $O(n+e)$ ✓

Adj list size → $n+e$

con n

undirected $\frac{n(n-1)}{2} \rightarrow e$

directed $n(n-1) \rightarrow e \rightarrow O(n^2)$

Matrix ✓ size → $O(n^2)$

Adj. List × size → $n + O(n^2)$

→ visiting the node of tree/graph in a particular order & processing the information only-once.

Graph Traversals (Techniques)

Tree Traversal vs Graph Traversal.

→ is always unique.
→ it starts from root.

→ is not unique.
→ it starts from any node.

(i) Depth First Search (DFS)

a) Status of a Node during Traversal
1) E-node : currently being explored
2) Live Node : all of whose children has not been generated. (LIFO stack)
3) Dead Node : fully explored node.

b) Timing values associated with Nodes, during Traversal.

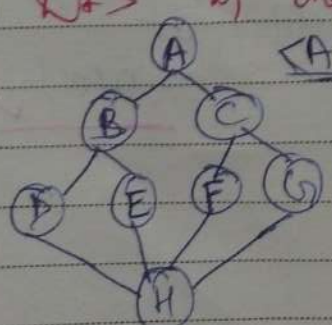
(x) d → discovery time
f → finishing time

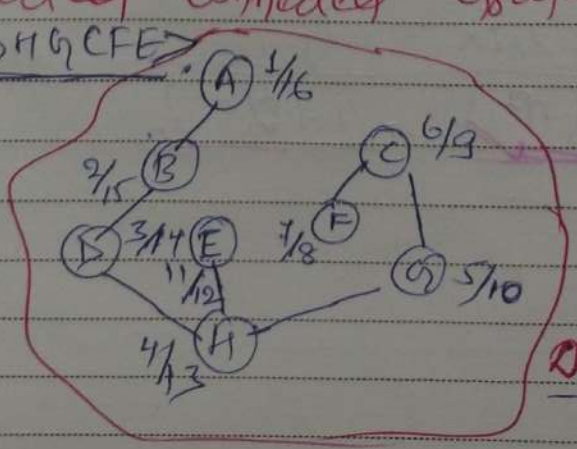
(integer values)

d: discovery time
: time at which node is visited for first time.

f: finishing-time
: time at which node becomes

(1) DFS in undirected connected Graph.



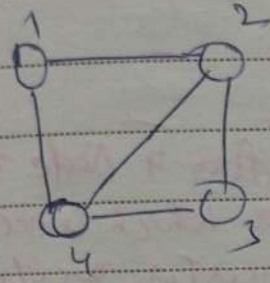


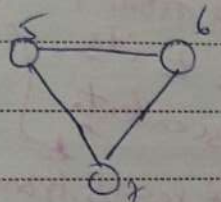


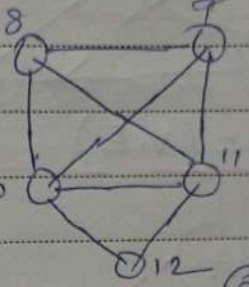
DFS spanning tree

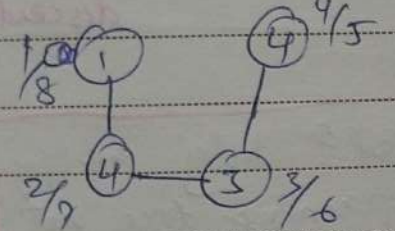
2) DFS in undirected Disjoint Graphs (Disconnected)

DFS when performed on undirected disjoint graphs is known as DFT (it will lead to multiple spanning trees)
(↳ Depth first traversal)

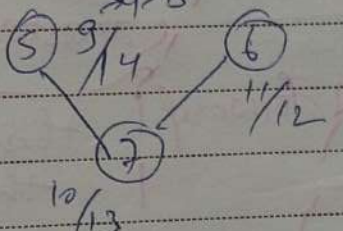




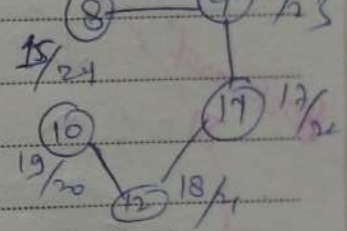




Spanning Tree 1
(ST_1)



(ST_2)



(ST_3)

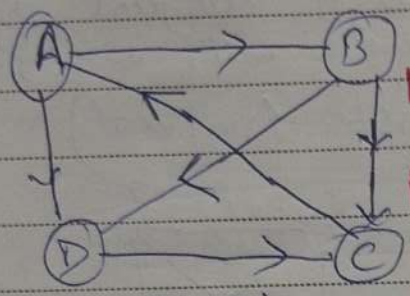
DFS Spanning Forest

Q Let the Graph has 4 vertices <P Q R S>

	P	Q	R	S	
a)	<3,14>	<4,12>	<8,10>	<15,18>	: (P Q R) (S) 2-components
b)	<14,20>	<5,12>	<8,10>	<15,16>	: (Q R) (P S) 2-components
c)	<2,40>	<8,10>	<5,20>	<45,50>	: (S) 4-components.
d)	<10,14>	<8,12>	<3,15>	<8,22>	: Invalid traversal

(P) (Q) (R)

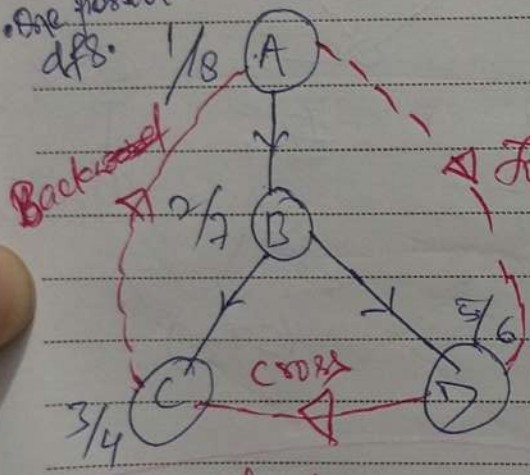
(3) DFS in directed Graphs: 4 Types of Edges.



$G = (V, E)$

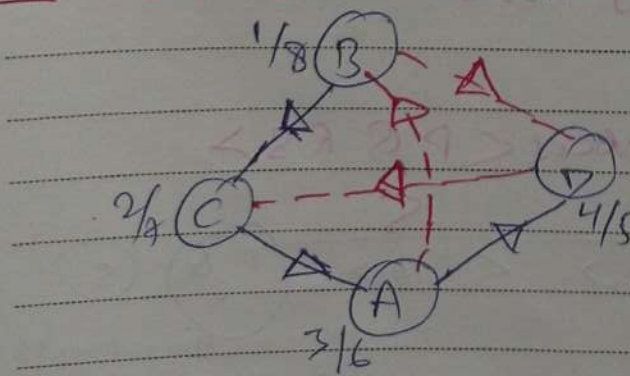
a) Tree - Edge → part of DFS sp. tree.
b) Forward edge → leads from a Node to its descendant.
c) Back edge → it leads from a Node to its ancestor.
d) Cross edge → Lead from a Node to another which is neither its ancestor nor its descendant.

• One possible dfs.



• these def² only apply in the spanning tree not in graph.

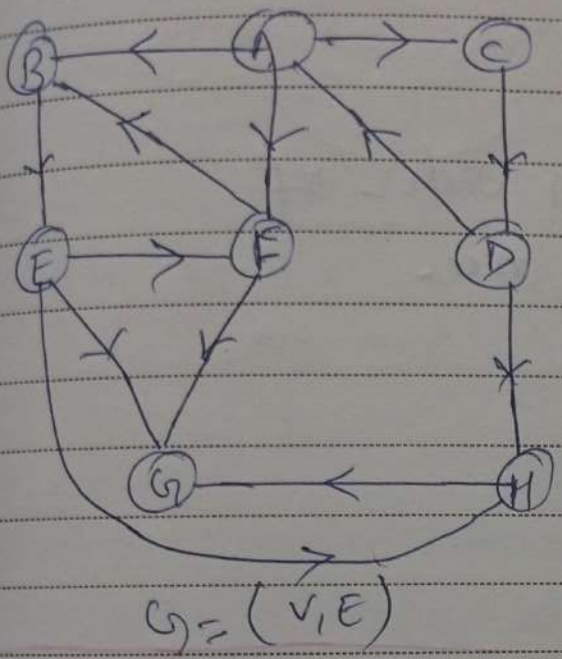
Sp. tree



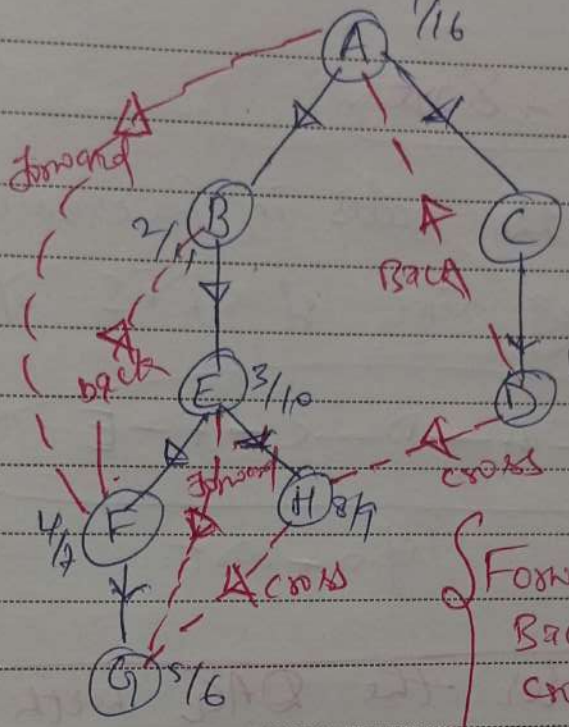
(A, B) → Back
(B, D) → Forward
(D, C) → Back.

↓ not sibling as C → A → D
grandchild

* directed graph



• Valid DFS traversal order.



Forward edge = 2
Back " = 2
Cross " = 2

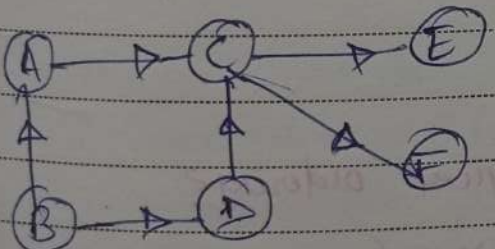
4) DFS in D.A.G <directed acyclic graph>

↓

Topological sort → (linear order)

↓

(start from that vertex which does not have any incoming edge)

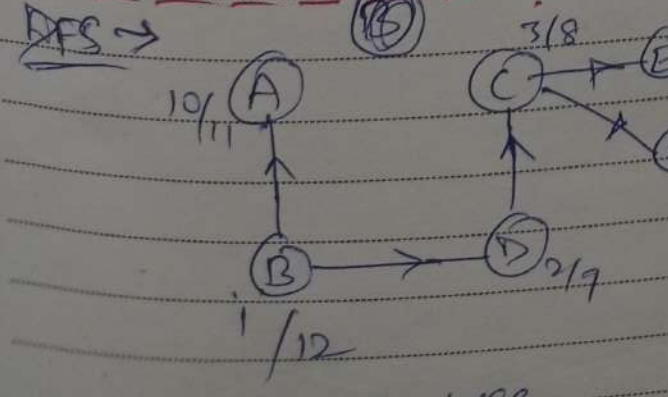


D.A.G

B
- A — D — C — E — F — ①
- A — D — C — F — E — ②
- D — A — C — E — F — ③
- D — A — C — F — E — ④

4-topological ordering

DFS →



Sp. tree

* We can get topological ordering from DFS.

- Parenthesization theorem based on DFS finishing time.

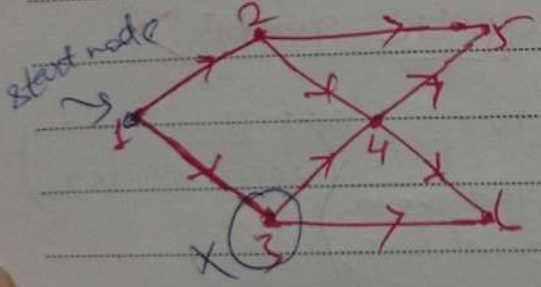
Topo-Sort.

Arrange the nodes in decreasing order of finishing time of DFS sp. tree.

B — A — D — C — F — E

Topo-Sort

To Q Consider the DAG with V = {1, 2, 3, 4, 5, 6}, shown below



Which of the following is NOT a topological ordering?

a) 1 2 3 4 5 6
b) 1 3 2 4 6 5
c) 1 3 2 4 5 6
d) 3 2 4 1 6 5

DFS → Components

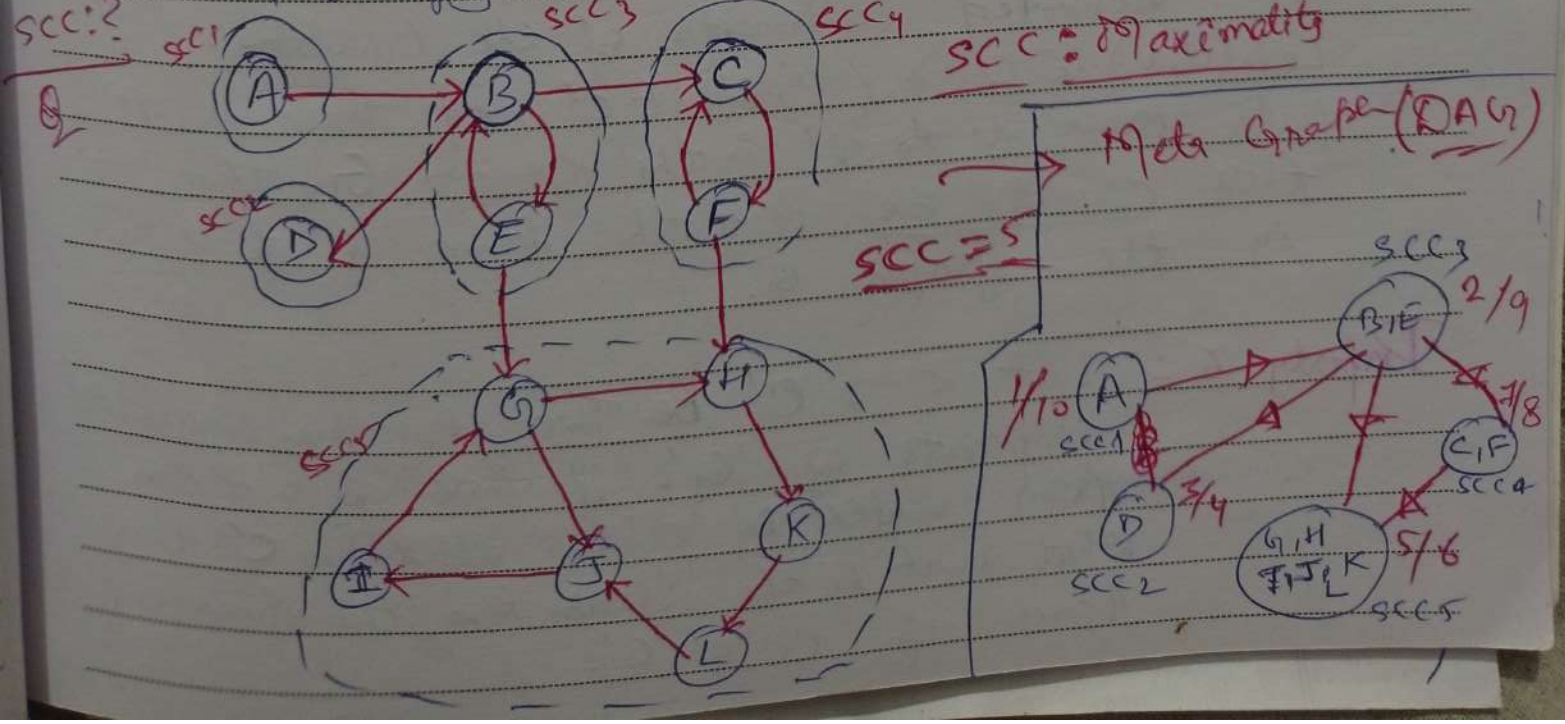
→ applicable to undirected graphs : Maximal connected subgraph

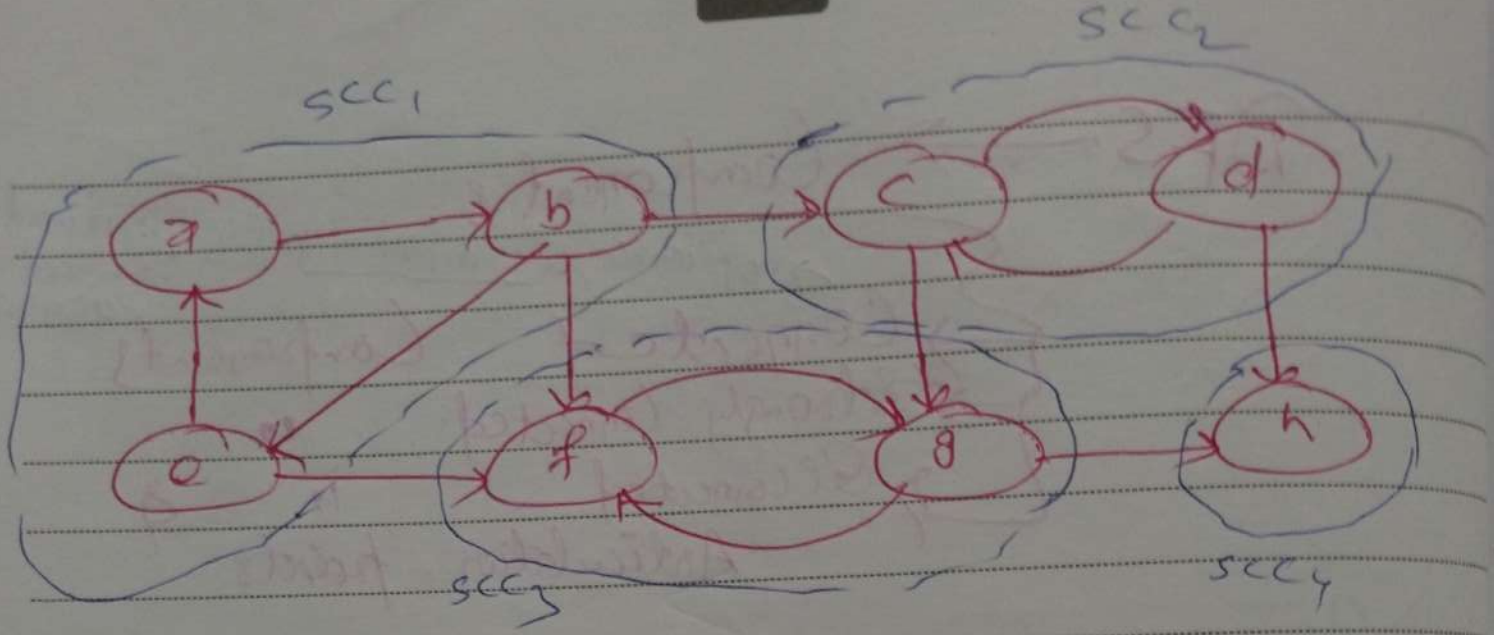
- Connected Components
- Strongly Connected "
- Biconnected " & Articulation points

Strongly Connected Components :

→ Two vertices 'u' & 'v' in a directed graph are said to be connected iff (if & only if) there is a path from 'u' to 'v' & also a path from 'v' to 'u'. They such two vertices in directed graphs are said to be connected.

→ This relation partitions the vertex set 'V' of the graph into disjoint sets known as Strongly Connected Components.





• Total SCC = 4

Vertices → {a, b, e} {c, d} {f, g} {h}

→ there should be no common vertex (each should be disjoint set.)

Property 1 :- Every directed graph is a directed acyclic graph of its strongly connected components.

Property 2 :- Let C & C' be two distinct strongly connected components of the Graph. Let (u, v) ∈ C & (u', v') ∈ C', if there is a path from u to v' in the graph G, then there cannot be a path from v' to u in the graph G.

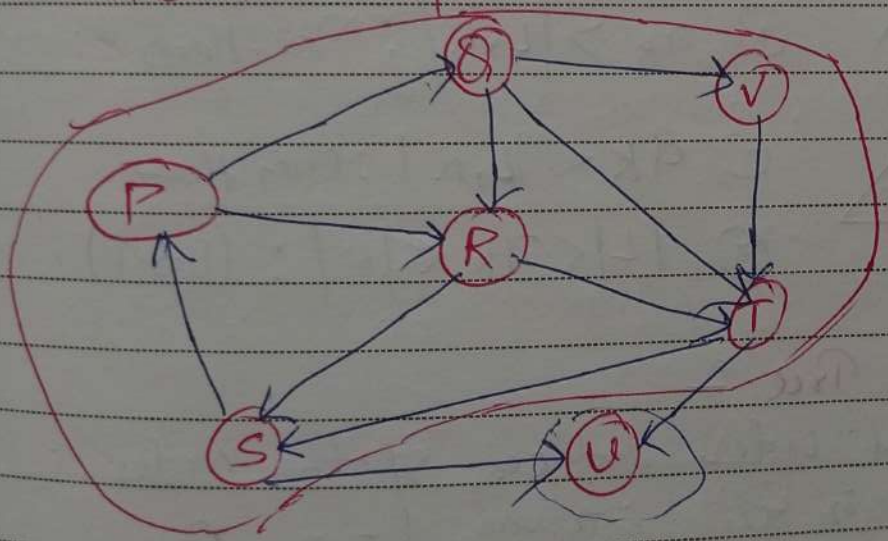
Property 3 :- Let C & C' be two strongly connected components of 'G'. if there is a path from vertex in 'C' to vertex in C' then what must be the relation b/w the finishing Times of C & C'.

$f(c) < f(c')$ ✗ [highest finishing time] → f

$f(c) > f(c')$

$f(c)$ any relation with $f(c')$ ✗

Q Which of the following is the correct decomposition of the directed graph given below into its strongly connected components



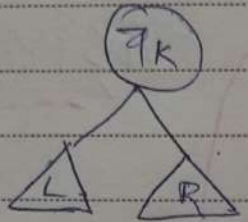
~~{P,Q,R,S,T,V},{U}~~

Heaps :- It is a complete Binary Tree with the property that the value at each node is greater (smaller) than the values of the nodes in the Left & Right Subtrees.

(Implement ⇐ Priority Queue)

(Implementation uses ⇒ Binary Tree)

Heap < max / min



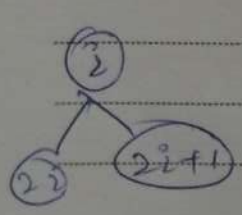
① $9_K > |L, R|$: Max-Heap

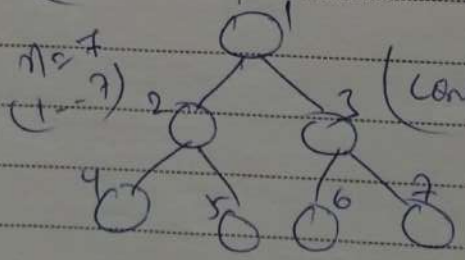
② $9_K < |L, R|$: Min Heap

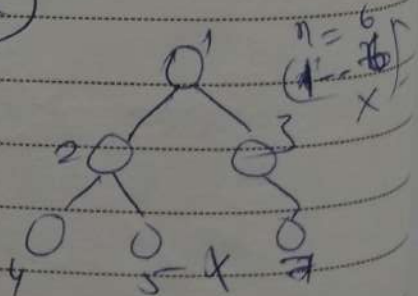
③ $|L| < 9_K < |R|$: (B.S.T)

Complete Binary Tree :-

The Level order Indices of the Nodes must lie in the interval 1 - - - n.
(n = no. of Nodes in the Tree)







Heap Creation :-

Insertion Method

→ We construct Heap from scratch from empty tree.

Max-heap

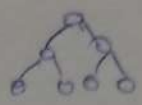
Heapify (Build-Heap)

↓

process of converting B.T to Heap.

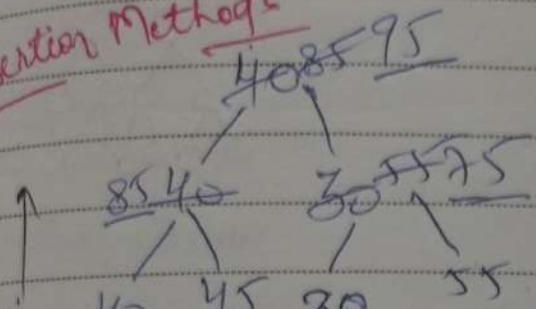


• Full-Binary Tree ⇒ max 2^h no. of nodes at each level



A: < 40, 85, 30, 75, 45, 55, 75 >

*Insertion Method:



• Time Complexity to construct heap from scratch.

① Best-Case: Decreasing order
↳ O(n)

② Worst-Case: Increasing Order/ASC
↳ O(n.Log n)

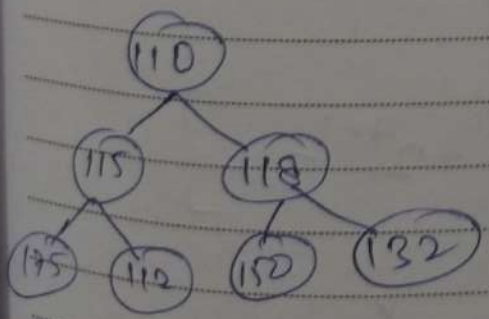
*Inserting a element in heap → Given a heap with n-elements, the Time Complexity to insert a new-element into it is

B.C → O(1), W.C → O(Log n)

*Heapify method

A: < 110, 115, 118, 175, 112, 150, 132 >

	1	2	3	4	5	6	7	
A	110	115	118	175	112	150	132	CBT
A	175	115	150	110	112	118	132	Heapify

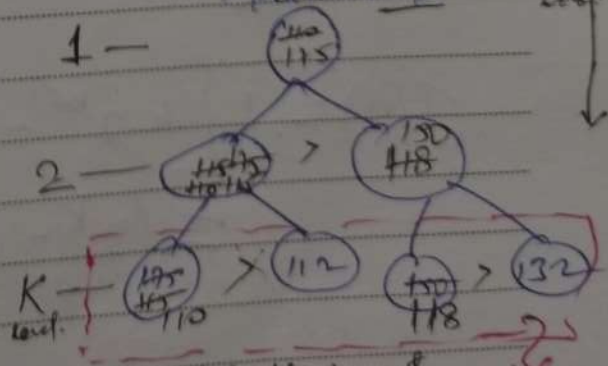


C.B.T

Heapify / Build-Heap

Max-Heap

top-bottom



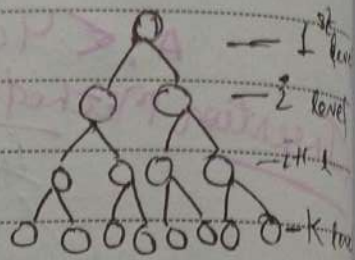
→ no. need of adjustment at leaf level.

(n/2 approx node at leaf level)

Time Complexity of Heapify method - O(n)

Time Complexity of Heapify Method ?

→ Max. No. of Nodes in a B.T at level 'i' : 2^{i-1}



Comps of i = k-i

→ Max. No. of Level Mov's for adjusting a Node at level 'i' is : $(k-i)$

→ Total Level movement for all Nodes at level i : $(k-i) \cdot 2^{i-1}$

→ Time Complexity, $T(n) = \sum_{i=1}^{k} (k-i) \cdot 2^{i-1}$

$$T(n) = \sum_{i=1}^{k} (k-i) \cdot 2^{i-1}$$

$$= \frac{1}{2}\left[\sum_{i=1}^{k} k \cdot 2^{i} - \sum_{i=1}^{k} i \cdot 2^{i}\right]$$

$$= \frac{1}{2}\left\{k\left(2^{k+1}-2\right) - \left((k-1)\cdot 2^{k+1}+2\right)\right\}$$

$$T(n) = \frac{1}{2}\left\{k \cdot 2^{k+1} - 2k - k \cdot 2^{k+1} + 2^{k+1} - 2\right\}$$

$$= \left(2^{k} - k - 1\right)$$

$\because n = 2^k \Rightarrow k = \log n$ (n → No. of nodes in tree)

$$= n - \log n - 1 = O(n) = \text{T.C of Heapify}$$

$$\sum_{i=1}^{n} i \cdot 2^{i} = (n-1)\cdot 2^{n+1} + 2$$

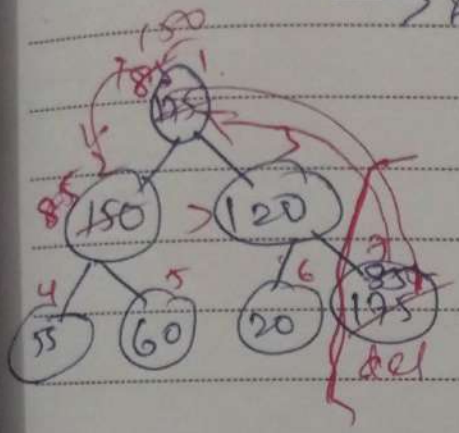
$$\sum_{i=1}^{n} 2^{i} = 2^{n+1} - 2$$

* Delete-op^2 in Heap

↳ Deleting (Root) {

1. Swap (A[1], A[n]); → O(1)
2. Adjust (A[1]);

} → O(Log n)



T.C for del-op^n → O(Log n)

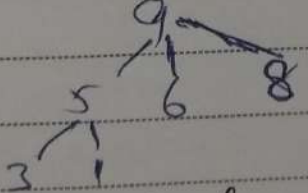
* Inc/Dcr (key) / Modify key.

↳ T.C → O(Log n)

* T.C of Heap sort → O(n Log n)

Q 3-ary max^2 heap array representation

<9, 5, 6, 8, 3, 1>



Q In binary max-heap with n ele, the smallest-ele can be found in time of _____.

→ Convert max-heap to min-Heap : O(n)